

Le Testeur Agile



Contenu

Module 1 - Introduction	5
1.1 Objectifs du module	5
1.2 L'agilité en chiffres	5
Les raisons de l'échec	6
Notions de fonctionnalités	6
Pourquoi mettre en place l'agilité ?	7
Maturité des pratiques agiles	7
La promesse des méthodes agiles	8
Les principales raisons d'échec	9
1.3 Historique des méthodes agiles	9
1.4 Comprendre les valeurs et les principes agiles	10
1.5 Glossaire	13
Module 2 - Les bases de Scrum	15
2.1 Objectifs de module	15
2.2 Le guide Scrum	16
2.3 Méthode Scrum	17
2.4 Rôles et engagements	19
Le Produit Owner	19
Le Scrum Master	19
L'équipe Scrum	20
Les rôles	20
2.5 Sprint	21
Vision du produit et découpage fonctionnel	22
Planification du SPRINT & RITUELS	23
2.6 User Story	23
L'apport de Scrum	24
Modèles pour les user stories	25
Le concept des 3 C	25
Propriétés des User Stories	26
Principe de validation	26
Exemples de User Stories	27
2.7 Critères d'acceptation	27
Rédaction	29

Exemples de spécification de critères d'acceptation	29
2.8 Planification	30
Planification du SPRINT	30
Itérations et incréments	31
2.9 Artefacts	31
Le Backlog produit	31
Le Backlog de sprint.....	32
Les critères < Done > (terminé)	32
2.10 Release Planning.....	33
Planification de releases	34
L'estimation de l'effort	34
Le Panning Poker	34
2.11 Sprint Planning	35
L'objectif du sprint	35
Sprint Backlog	36
Les règles	37
2.12 Suivi du Sprint.....	37
Le Daily Scrum (ou standup).....	37
Kanban Board	38
Le Daily Scrum- les règles	39
Burndown Chart	39
Burndown chart de la release	40
La revue de sprint	41
2.13 Rétrospective.....	42
Facteurs clés de succès pour les rétrospectives	42
Exemple – rétrospective hebdomadaire sprint n° 03	42
Module 3 - Conduite du changement	45
3.1 Objectifs du module	45
3.2 Culture	45
La rupture culturelle	45
Le modèle de Schneider	46
Les questions posées par l'agilité	46
Les valeurs fondamentales	47
3.3 Conduite du changement	50
3.4 Gemba	51
Conduite du changement	51

Objectif du Gemba	52
La marche du Gemba.....	53
3.5 Références pour aller plus loin	53
Atelier 1 : Le testeur à la découverte du monde Agile	54
Contraintes	54
Accéder à l'activité suivante, et prenez en charge ces tâches.....	54
Codes couleurs	55
Atelier 1	55
Correction Atelier 1	56
Atelier 2 - La qualité c'est pour tout le monde	56
Contraintes	56
Lors de la création d'un mode d'emploi de construction du produit,	57
Ecrivez dans un fichier un mode d'emploi de construction du produit :	57
Atelier 2 : Correction	58
Atelier 3 : Multiplier les exécutions	60
Contraintes	60
Atelier 3	61
Résultat de l'activité	61
Atelier 3 : Bilan	61
Atelier 4 : Importance de la communication	62
Contraintes	62
Répartition des rôles	63

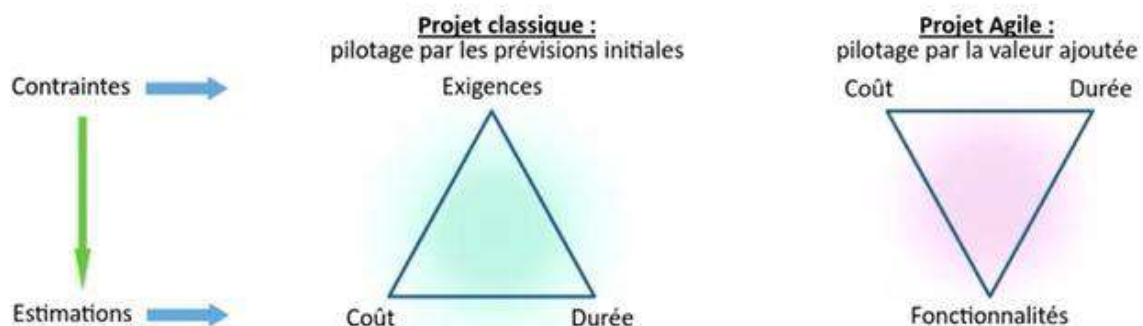
Module 1- Introduction

1.1 Objectifs du module



1.2 L'agilité en chiffres

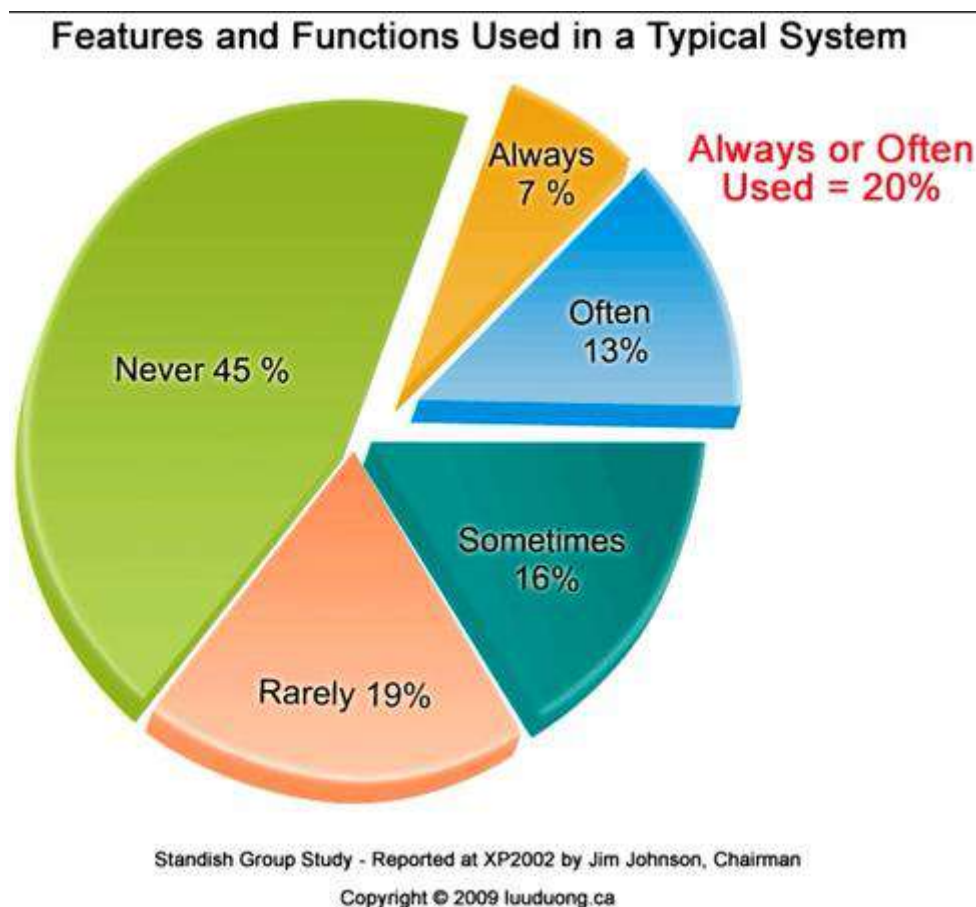
"L'Agilité est la capacité à favoriser le changement et à y répondre pour s'adapter au mieux à un environnement turbulent" *Jim Highsmith (auteur Agile)*



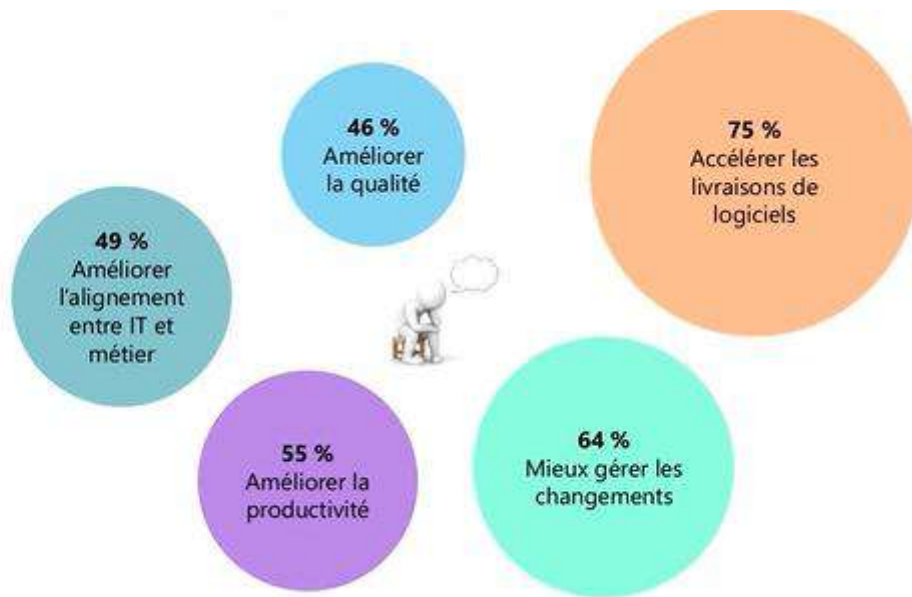
Les raisons de l'échec



Notions de fonctionnalités



Pourquoi mettre en place l'agilité ?



Maturité des pratiques agiles



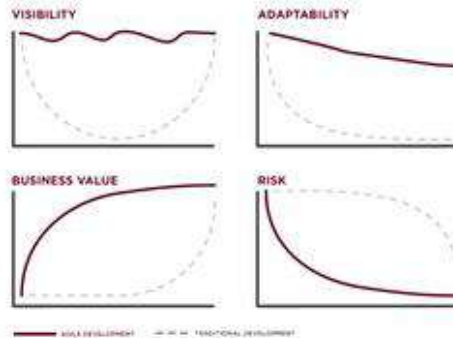
- La quasi-totalité des sociétés a aujourd'hui mis en œuvre des pratiques agiles dans ses projets.
- Une grande majorité les applique depuis plusieurs années.
- Mais une petite minorité se considère comme « mature ».

La promesse des méthodes agiles

Rendre plus visible l'avancement du projet, les interactions et la communication permanente dans l'équipe et partagée avec les clients.

Fournir en priorité ce qui a le plus de valeur, ce qui est le plus important pour le client, produisant un logiciel opérationnel très tôt.

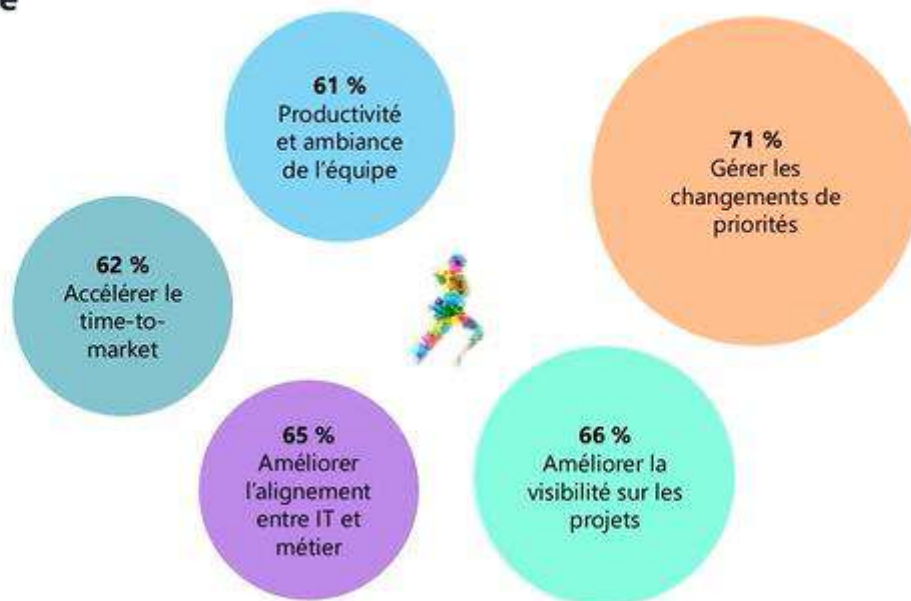
AGILE DEVELOPMENT VALUE PROPOSITION



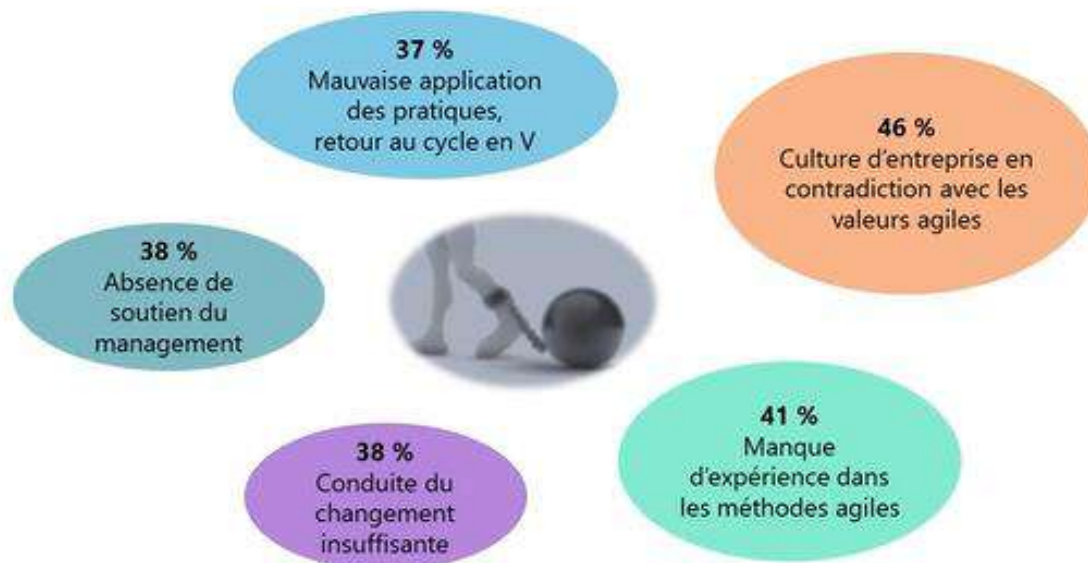
Appliquer une démarche itérative, incrémentale et adaptative facilitant la réactivité face au changement.

Faire des erreurs tôt et avoir un feedback rapide pour corriger à moindre coût.

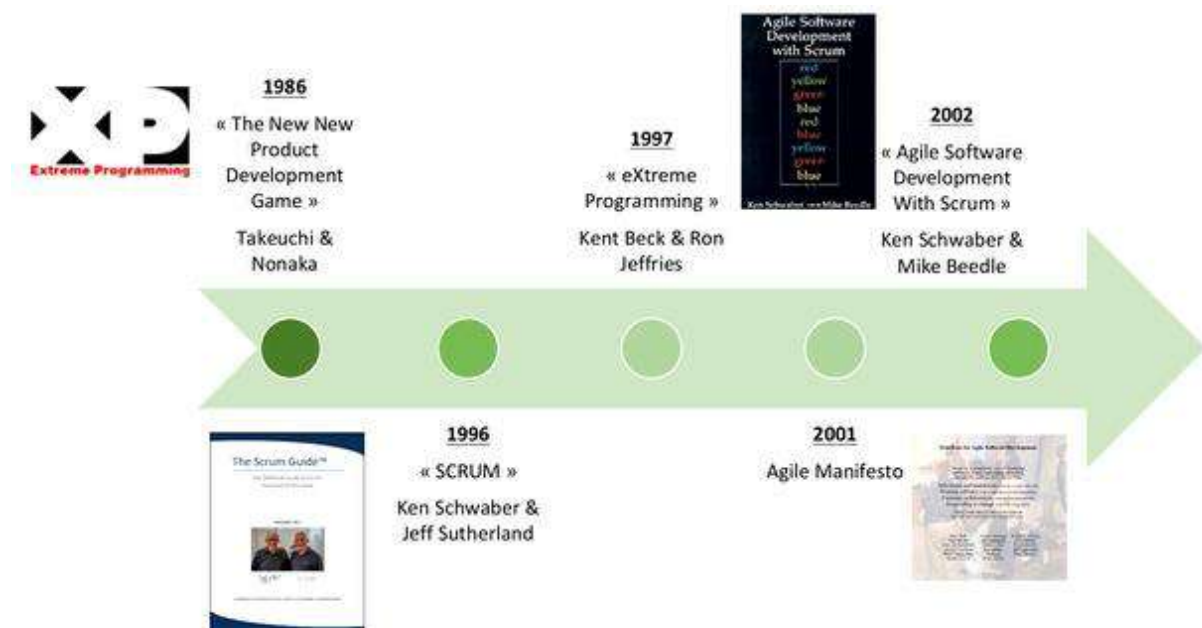
... et la réalité



Les principales raisons d'échec



1.3 Historique des méthodes agiles



1.4 Comprendre les valeurs et les principes agiles

- Le manifeste agile et les valeurs de l'agilité
- Panorama des méthodes agiles
 - eXtreme Programming
 - Scrum
 - Kanban
 - Lean
- Vision du produit et découpage fonctionnel
- L'équipe intégrée et les règles du jeu

Manifeste Agile

Les 4 valeurs du développement Agile :

- Individus et échanges plus que processus et outils.
- Produit fonctionnel plus que documentation pléthorique.
- Collaboration du client plus que négociation du contrat.
- Réactivité au changement plus que suivi d'un plan.

Les 12 principes du développement Agile :

Notre priorité première est de satisfaire le client en livrant au plus tôt et de manière constante un logiciel de qualité.

Tout changement, même tardif, des exigences pendant le développement est bienvenu. Les méthodes Agiles transforment le changement en avantage compétitif pour le client.

Livrer régulièrement un logiciel fonctionnel, toutes les deux semaines à deux mois, en préférant la plus petite périodicité.

Maîtrise d'ouvrage et développeurs doivent collaborer quotidiennement tout au long du projet.

Bâtir le projet avec des personnes motivées. Leur donner l'environnement et le soutien dont elles ont besoin et croire en leur capacité à accomplir le travail.

La plus efficace des méthodes pour transmettre l'information au sein et à destination d'une équipe est le face à face.

Un logiciel qui fonctionne est le meilleur indicateur de progrès.

Les méthodes Agiles favorisent un rythme de développement soutenable. Commanditaires, développeurs et utilisateurs devraient pouvoir maintenir ce rythme indéfiniment.

Une attention constante à l'excellence technique et à la qualité de la conception améliore l'agilité.

La simplicité - l'art de maximiser la quantité de travail à ne pas faire - est essentielle.

Les meilleures architectures, spécifications et conceptions sont issues d'équipes auto-organisées.

À intervalle régulier, l'équipe réfléchit aux moyens de devenir plus efficace, puis modifie et ajuste son comportement dans ce sens.

D'après Dilbert...



© Scott Adams, Inc./Dist. by UFS, Inc.

Nous allons essayer une chose qui s'appelle développement agile.

Cela veut dire plus de planification et plus de documentation. Commencez juste à coder et à vous plaindre.

*- Je suis content que ça porte un nom.
- Voilà, c'était votre formation.*

Et vous, quelle est votre expérience de l'agilité ?



1.5 Glossaire

Terme original	Traduction	Description
Backlog	Liste	Liste ordonnée et priorisée de toutes les choses à faire. Il y a le backlog de produit qui énumère les exigences avec le point de vue du client et le backlog de sprint qui contient les tâches de l'équipe.
Burndown chart	Graphique d'avancement	Représentation graphique du reste à faire dans une période, actualisé aussi souvent que possible et permettant de montrer une tendance de l'avancement. Ce graphique montre la tendance du reste à faire total de jour en jour (pour les sprints) ou de sprint en sprint (pour les releases).
Burnup chart	Graphique d'avancement	Il permet de suivre l'évolution du travail terminé. Le but consiste donc à atteindre le plus tôt possible le niveau le plus haut, d'où le terme « Up ». Il est souvent utilisé comme graphique d'avancement d'une Release.
Daily Scrum	Mêlée quotidienne	Réunion quotidienne de 15 minutes qui a pour but de faire le point sur ce qui a été fait depuis la dernière mêlée, ce qui est prévu de faire jusqu'à la prochaine et quelles sont les embûches rencontrées durant le travail.
Definition of Done	Définition de terminé	Définition donnée par l'équipe servant à déterminer quand la réalisation d'une tâche peut être considérée comme terminée
Epic	Epopée	Il s'agit d'un ensemble de user stories, qui est assimilé le plus souvent à une fonctionnalité que doit fournir le produit
Extreme Programming (XP)		Référentiel de bonnes pratiques de développement pouvant être utilisées dans un contexte agile (légère) car adaptée aux équipes réduites avec des besoins changeants, centrée sur les tests et proche du client. Elle pousse à l'extrême des principes simples

Terme original	Traduction	Description
Lean IT		Le lean IT est l'extension des principes du Lean au monde des systèmes d'information (développement, management de projets informatiques, infrastructures...). L'idée directrice, ici appliquée au monde des systèmes d'information, est l'élimination des pertes. On entend par pertes, le travail qui n'apporte aucune valeur métier à un produit ou à un service.
Planning poker		Atelier d'équipe servant à estimer les points d'effort de chaque User Story, en utilisant un jeu de cartes composé de la suite de Fibonacci
Potentially shippable increment	Produit potentiellement utilisable	Incrément livré par l'équipe en fin de sprint, démontré au client et potentiellement utilisable ou pouvant être déployé en production
Product Backlog	Périmètre produit	Liste des fonctionnalités qui devront être réalisées par le logiciel
Product owner	Responsable produit	Personne responsable de produire et maintenir à jour le backlog de produit. C'est lui qui en détermine les priorités et qui prend les décisions concernant l'orientation du projet.
Release	Version	Une release correspond à la livraison d'une version. Par habitude, on parle de release pour considérer la période de temps qui va du début du travail sur cette version jusqu'à sa livraison et qui passe par une série de sprints successifs.
Release Planning	Planification de release	Période de temps allouée en début d'un projet afin de définir et planifier le contenu de plusieurs sprints. En général, le Product Owner construit le product backlog (epics et user stories du produit) et le présente à l'équipe. Les user stories sont complétées, estimées et planifiées sur les premiers sprints.

Terme original	Traduction	Description
Retrospective	Rétrospective	A chaque fin de sprint il y a des réunions de travail, appelées les rétrospectives. C'est l'occasion pour l'ensemble de l'équipe de faire le bilan sur les points positifs et négatifs du sprint passé. Chacun a alors la totale liberté d'exprimer ce qu'il ressent.
SCRUM		Scrum est un démarche agile de développement de logiciels qui propose une organisation du projet itérative et incrémentale
Scrum Master		Membre de l'équipe dont l'objectif principal est de faciliter le travail de l'équipe et la protéger des perturbations extérieures. Il est complètement transparent pour la communication entre l'équipe et les clients et n'a aucun pouvoir hiérarchique sur l'équipe. Il s'assure également que les principes Scrum sont bien assimilés et appliqués par l'équipe
Sprint	Itération	Nom d'une itération dans Scrum. Cette itération dure 30 jours calendaires en théorie, mais en pratique on utilise plutôt entre 2 et 4 semaines. Pendant une itération, l'équipe doit développer une liste d'items du backlog de produit qui a été définie au début de ce sprint.
Sprint Backlog	Périmètre de l'itération	Liste des tâches à accomplir pendant un sprint. Elles correspondent à la réalisation des items de backlog du produit affectés au sprint.
Sprint Planning	Planification d'itération	Réunion d'équipe permettant de définir le périmètre du Sprint. Le Product Owner présente l'objectif, à savoir ce qu'il souhaite faire construire durant le sprint. L'équipe estime la complexité et s'engage sur ce qu'elle peut produire (sprint backlog)
Sprint Review	Revue de sprint	Réunion pendant laquelle l'équipe montre le résultat de son travail au Product Owner et aux parties prenantes, ce qui permet d'avoir un retour sur ce que l'atteinte de l'objectif du sprint et l'adéquation par rapport au besoin

Terme original	Traduction	Description
Story points	Points d'effort	Dans les projets agiles on définit des points d'effort. Une échelle de points permet d'évaluer les difficultés de développement à partir des critères suivants : le temps de réalisation, la difficulté technique, la difficulté des tests clients.
Taskboard (ou Kanban)	Tableau des tâches	Représentation visuelle des tâches que doit réaliser l'équipe pendant le sprint. Il contient par défaut trois colonnes « A faire », « En cours » et « Terminé ». Le tableau est mis à jour par l'équipe à chaque daily.
User story	Récit utilisateur	Phrase simple dans le langage de tous les jours permettant de décrire avec suffisamment de précision le contenu d'une fonctionnalité à développer. La phrase contient généralement trois éléments descriptifs de la fonctionnalité : qui ? Quoi ? Pourquoi ?
Velocity	Vélocité d'équipe	En Scrum, on appelle « vélocité » le nombre de points d'effort « terminés » au cours d'un sprint. Il s'agit d'un indicateur qui a pour but d'aider l'équipe de développement lors des séances de planification de sprint à évaluer la quantité de travail qu'elle peut sélectionner pour le sprint. Cet indicateur peut être utilisé par le Product Owner pour l'aider à planifier les livraisons.

Module 2- Les bases de Scrum

2.1 Objectifs de module

Comprendre les bases
de SCRUM

Identifier les composantes
de SCRUM :

- Rôles
- Rituels
- Artéfacts

Plannifier suivant SCRUM

- Release planning
- Sprint planning
- Suivi du sprint
- La rétrospective

2.2 Le guide Scrum

Définition de Scrum

Scrum est un cadre de travail léger qui aide les personnes, les équipes et les organisations à générer de la valeur grâce à des solutions adaptatives pour des problèmes complexes.

En bref, Scrum a besoin d'un Scrum Master pour favoriser un environnement où :

1. Un Product Owner ordonne le travail à faire pour résoudre un problème complexe dans le Product Backlog.
2. La Scrum Team transforme une sélection de ce travail en un Increment de valeur lors d'un Sprint.
3. La Scrum Team et ses parties prenantes inspectent les résultats et s'adaptent pour le prochain Sprint.
4. *Répéter*

Scrum est simple. Essayez-le tel qu'il est et, déterminez si sa philosophie, sa théorie et sa structure aident à atteindre les objectifs et à créer de la valeur. Le cadre de travail Scrum est volontairement incomplet, ne définissant que les parties nécessaires pour mettre en œuvre la théorie Scrum. Scrum repose sur l'intelligence collective des personnes qui l'utilisent. Plutôt que de fournir aux gens des instructions détaillées, les règles de Scrum guident leurs relations et leurs interactions.

Divers processus, techniques et méthodes peuvent être employés dans ce cadre de travail. Scrum englobe des pratiques existantes ou les rend inutiles. Scrum rend visible l'efficacité relative du management existant, de l'environnement et des techniques de travail, afin que des améliorations puissent être apportées.

Théorie Scrum

Scrum est fondé sur l'empirisme et la pensée Lean. L'empirisme affirme que la connaissance provient de l'expérience et que la prise de décision s'appuie sur l'observation de faits. La pensée Lean réduit le gaspillage et se focalise sur l'essentiel.

Scrum utilise une approche itérative et incrémentale pour optimiser la prédictibilité et le contrôle de risque. Scrum engage des groupes de personnes qui ont collectivement toutes les compétences et l'expertise requises pour faire le travail et partager ou acquérir de telles compétences selon les besoins.

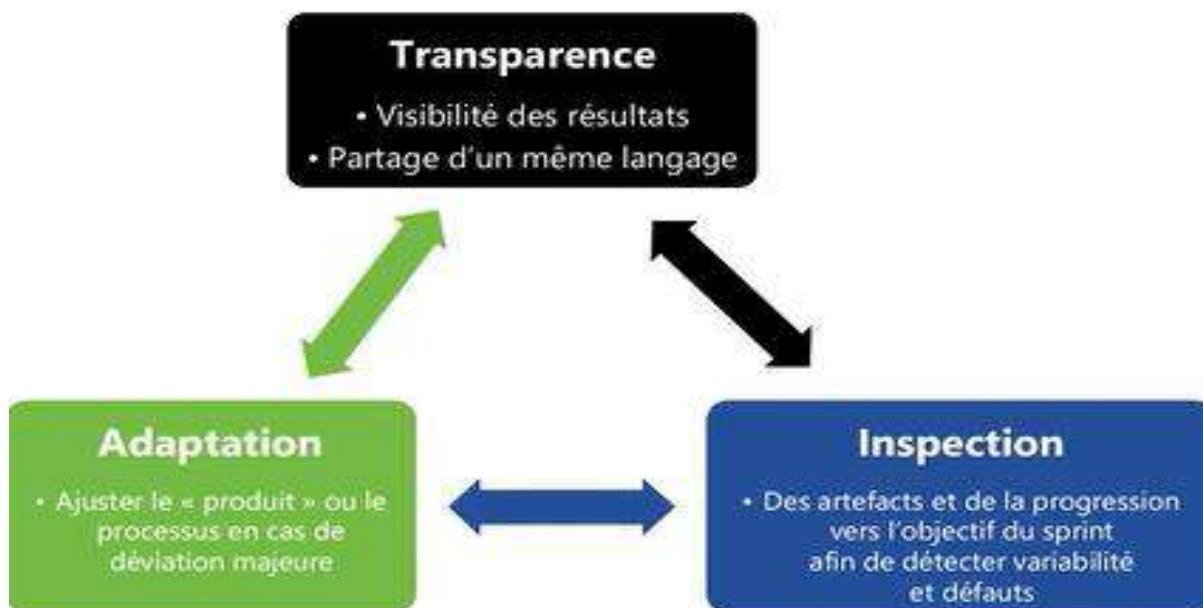
Scrum combine quatre événements formels pour l'inspection et l'adaptation dans un événement conteneur, le Sprint. Ces événements fonctionnent parce qu'ils mettent en œuvre les piliers empiriques de Scrum de transparence, d'inspection et d'adaptation.

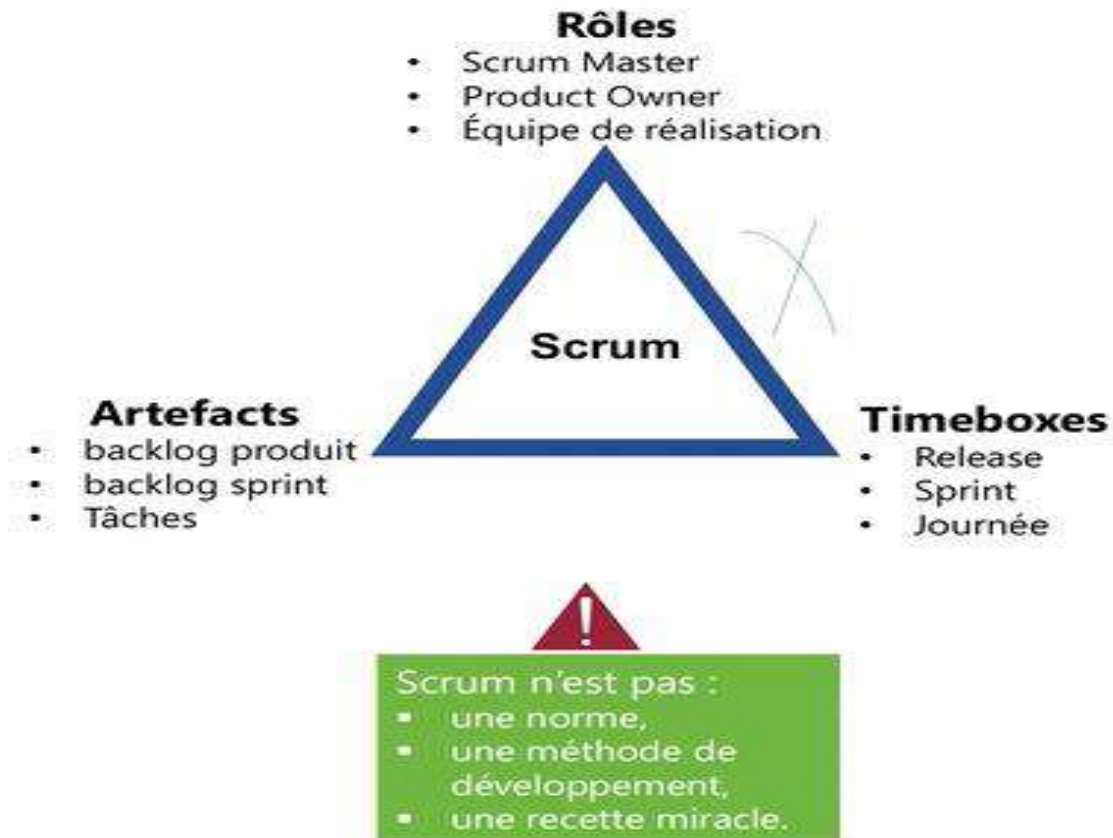
Transparence

Le processus et le travail émergents doivent être visibles pour ceux qui effectuent le travail ainsi que pour ceux qui le reçoivent. Avec Scrum, les décisions importantes sont fondées sur l'état perçu de ses trois artefacts formels. Des artefacts peu transparents peuvent mener à des décisions qui diminuent la valeur et augmentent le risque.

2.3 Méthode Scrum

- Les nouveaux rôles dans l'équipe : Scrum Master et Product Owner
- Le découpage temporel du projet (*timeboxing*) : releases et sprints
- Le découpage incrémental du produit : backlog, epics et user stories
- Les rituels de l'équipe agile





- Les rôles de l'équipe
- Les itérations : « timeboxing »
- Les incréments
- L'approche empirique
- La valeur métier
- La planification multiniveau



2.4 Rôles et engagements

Le Produit Owner

Le Product Owner

- Le Product Owner (PO)
 - génère et maintient le backlog du produit,
 - fournit la vision globale du produit,
 - priorise les fonctionnalités du backlog,
 - s'assure que chaque élément est compris par l'équipe,
 - est garant du retour sur investissement du projet,
 - représente les clients et utilisateurs,
 - peut valider les fonctionnalités livrées,
 - peut communiquer sur l'avancement du produit.



Il **s'engage** à définir la vision du produit, à prioriser le backlog du produit et à prendre des décisions sur ce qui doit être développé.



Le Product Owner doit être :

- légitime et avoir assez de pouvoir pour trancher,
- disponible et impliqué sur le projet.

Le Scrum Master

Le Scrum Master

- Le Scrum Master (SM)
 - veille au bon déroulement du cycle de vie,
 - garantit le respect des règles par l'équipe,
 - résout les problèmes et blocages,
 - favorise l'autonomie de l'équipe,
 - facilite la communication et la collaboration,
 - protège l'équipe des perturbations extérieures.



Il **s'engage** à faciliter le processus Scrum, s'assure que l'équipe suit les principes Scrum, et élimine les obstacles.



Le Scrum Master n'est pas :

- un décideur,
- le responsable hiérarchique de l'équipe,
- un chef de projet qui définit les priorités de l'équipe.

L'équipe Scrum

L'équipe Scrum

- L'équipe Scrum (*the team*)

- Inter-fonctionnelle et responsable de :
 - Conception
 - Codage
 - Tests
 - Documentation

Auto organisée : l'équipe est autonome dans :

- La décision du périmètre à produire dans le sprint
- Le découpage du périmètre en tâches
- La répartition des tâches (auto-attribution)
- L'estimation de la complexité de réalisation
- La quantité de tests à effectuer

L'équipe Scrum s'engage à réaliser les tâches du backlog du sprint de manière autonome et à produire un incrément de produit fonctionnel à la fin de chaque sprint.



L'équipe Scrum doit toutefois respecter :

- Les priorités données par le PO
- Les règles du jeu définies avec le SM

Les rôles

Les rôles

- Carte d'identité d'une équipe intégrée

- Taille : petite (max 10 personnes)
- Adresse : espace de travail commun
 - Open space
 - Plateau de développement
- Signes particuliers
 - Réunions quotidiennes (*stand up meeting*)
 - Tableau et post-it de suivi des opérations

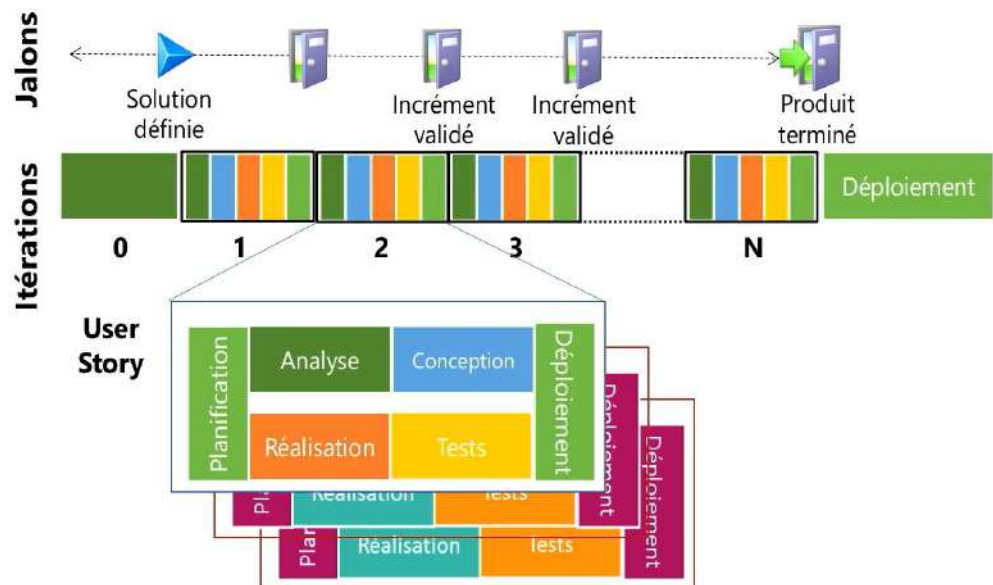


- Avantages pour l'équipe

- Meilleure communication et collaboration
- Capitalisation des différentes compétences au profit du projet
- Responsabilité partagée de la qualité

2.5 Sprint

Le sprint



- Le Sprint est le nom donné à l'itération dans Scrum
- C'est un temps pendant lequel l'équipe est focalisée sur son objectif : produire de la valeur !
- La durée d'un sprint est fixe
 - De 1 à 4 semaines
 - Au bout de la durée fixée, on arrête le travail
 - Pas de jours en plus pour terminer ce qui ne l'est pas
- La donnée d'entrée d'un sprint est le sprint backlog
 - L'engagement est pris par l'équipe (auto-organisation)
 - Toutes les fonctionnalités ont vocation à être finies
- A l'issue de chaque sprint :
 - L'équipe doit fournir un livrable, documenté et démontré (l'incrément).
 - L'incrément est une nouvelle version logicielle qui peut potentiellement être mise en production.

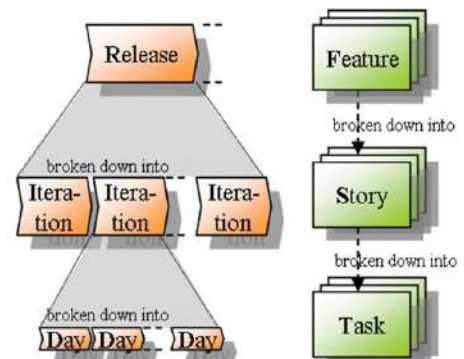
Les règles du sprint

- Une durée de 2 semaines facilite la prise en main de Scrum car elle donne de la visibilité au projet.
- La durée d'un sprint doit rester stable au cours du projet.
- Mais elle peut évoluer pour de bonnes raisons.
- Décomposer les tâches de façon à ce qu'elles durent quelques jours au maximum et fournir de la visibilité sur l'avancement à l'équipe.
- Le périmètre peut changer, pas la date !
- Les fonctionnalités peuvent être supprimées du sprint et replacées dans le backlog produit.
- L'équipe peut changer le périmètre du sprint : déprioriser ou sortir une story, en intégrer une nouvelle en provenance du Product Backlog...
- Mais c'est toujours le Product Owner qui aura le dernier mot !

Vision du produit et découpage fonctionnel

En contexte agile, la vision du produit est découpée en trois dimensions

- **Temps** (projet)
 - Releases : 3 à 6 mois
 - Itérations : 2 à 4 semaines
 - Journées
- **Produit**
 - Fonctionnalités : utile et ayant du sens pour un utilisateur
 - « User Stories » : élément testable d'une fonctionnalité
 - Tâches : activités élémentaires de travail (conception, développement, test...)
- **Équipe** : découpage des responsabilités
 - Équipe client
 - Équipe intégrée dev+test
 - Responsables



Planification du SPRINT & RITUELS

- Les différentes boîtes de temps (*timeboxes*)

Élément	Version originale	Caractéristiques	Durée
Planification de version	Release planning		Journée(s)
Planification d'itération	Sprint planning	Définir le contenu du sprint (<i>backlog</i>) et comment les fonctionnalités sélectionnées vont être développées	2 à 4 heures
Réunion quotidienne	Daily standup	Revue d'avancement des tâches réalisées	15 minutes
Itération	Sprint	Période fixe pendant laquelle l'équipe réalise le contenu	1 à 4 semaines
Revue d'itération	Sprint review	Démonstration au client des fonctionnalités développées et évaluation de l'atteinte des objectifs de l'itération	1 à 2 heures
Rétrospective	Retrospective	Analyse effectuée par l'équipe sur le déroulement de l'itération	1 à 2 heures

2.6 User Story

- Des mauvaises spécifications sont l'une des raisons majeures d'échec du projet
 - Manque de visibilité
 - Les utilisateurs ne savent pas toujours décrire ce qu'ils veulent
 - Peu de personnes dans l'équipe ont la vision globale de comment le système doit fonctionner
 - Problème de périmètre
 - Certaines fonctionnalités sont redondantes ou contradictoires
 - Problèmes de communication
 - La vision du produit et les fonctionnalités ne sont pas partagées dans l'équipe
 - Phénomène d'interprétation en cas d'ambiguïté des spécifications

- Solution pour les développements agiles = User Stories
 - Regroupent les points de vue
 - Clients, utilisateurs, métier via le PO
 - Développeurs et testeurs dans l'équipe
 - Regroupent les caractéristiques
 - Fonctionnelles : à quoi ça sert ?
 - Non fonctionnelles : comment ça fonctionne ?
 - Vision partagée des spécifications
 - Nombreuses revues informelles
 - Brainstorming, mind-mapping...
 - Critères d'acceptation systématiquement définis
 - *Definition of Done*

L'apport de Scrum

- Constat
 - 64 % des fonctionnalités développées sont peu ou pas utilisées
- L'objectif de la méthode est de délivrer avant tout ce qui a de la valeur
 - Mesure de la valeur créée pour l'utilisateur (ROI)
 - Mesure de productivité de l'équipe
- Le Product Owner
 - Définit une vision concernant le produit (épopée)
 - Décline cette vision en fonctionnalités
 - Décline chaque fonctionnalité en user stories
 - Priorise les fonctionnalités et user stories par la valeur qu'elles délivrent

Modèles pour les user stories

- Rôle : qui ?
 - Personne réelle
 - Utilisant la fonctionnalité
 - Avec un profil spécifique
- Action : quoi ?
 - Ce que la personne va faire
 - Décrit la fonctionnalité qui doit être implémentée
- Bénéfice : pourquoi ?
 - La raison pour laquelle on veut utiliser la fonctionnalité
 - Donne un sens à la fonctionnalité
 - Décrit ce qui est important pour l'utilisateur

En tant que **rôle**

Je souhaite effectuer une **action**

Afin d'en obtenir un **bénéfice**

Le concept des 3 C

- Carte
 - Ecriture de la user story sur une carte
 - Identifie la fonctionnalité à réaliser et sa criticité
 - Peut être annoté avec l'effort de réalisation (développement et test)
 - La description doit être précise, car elle sera utilisée dans le backlog produit
- Conversation(s)
 - Discussions entre l'équipe et le PO / client
 - Expliquent l'utilisation de chaque user story
 - Complètent la description avec des détails et des exemples
- Confirmation
 - Etablit des critères d'acceptation pour chaque user story
 - Propose des tests d'acceptation pour couvrir chaque critère d'acceptation
 - Prévoit des tests à la fois positifs et négatifs

Propriétés des User Stories

Propriétés des User Stories = INVEST	
Indépendante	Pas de recouvrement et pas de dépendance chronologique.
Négociable	Ne résulte pas d'un contrat mais d'une série de discussions entre l'équipe et le PO.
Valeur	Elle doit apporter de la valeur au client et donc être traitée verticalement, par exemple en mettant à disposition un traitement de bout en bout mais sur un cas spécifique seulement.
Estimable	Important pour fournir rapidement un niveau de priorité et une estimation de charge afin de pouvoir planifier sa réalisation.
Suffisamment petite	Représente quelques jours de travail pour le développeur, réalisable dans une itération.
Testable	Les critères d'acceptation sont identifiés, inclus dans la story et partagés avec l'équipe afin de faciliter la réalisation.

Principe de validation

- DOD – *Definition of Done*
 - Critères définis par l'équipe
 - Définissent la complétude du sprint
 - Couverture des fonctionnalités
 - Qualité obtenue
 - Dépendent de la compréhension
 - Des user stories
 - Des exigences client sur le produit
- Incrément produit
 - Résultat de sprint
 - Produit potentiellement livrable ou déployable en production
 - Faisant éventuellement l'objet d'une démo au client pour
 - Susciter des retours précoces
 - Etablir la confiance

Exemples de User Stories

En tant que client, je peux réserver une chambre d'hôtel.

En tant qu'agent de voyages, je peux voir des photos des hôtels.

En tant que client, je peux annuler une réservation

En tant que client, je peux appliquer des critères de recherche et ne voir que les hôtels avec des chambres disponibles.

2.7 Critères d'acceptation

- Que sont les critères d'acceptation ?
 - Les critères d'acceptation sont utilisés pour délimiter le périmètre des user stories (US).
 - Ils aident à expliciter les objectifs que l'acteur doit atteindre dans la US, en spécifiant pour chaque objectif les conditions de succès ou d'échec, sans toutefois spécifier la manière dont il doit s'y prendre.
 - Ces critères peuvent être définis sur plusieurs niveaux de détails : user stories, sprints, epics, releases...
- Pourquoi les écrire ?
 - Pour permettre la validation des user stories par le PO ou les testeurs.
 - Ils peuvent également être utilisés par les développeurs pour connaître les détails de l'implémentation de chaque user story.



- Qui doit les écrire ?

- Les critères d'acceptation devraient en priorité être rédigés par le rédacteur des US, à savoir le PO ou son représentant dans l'équipe.
- L'équipe doit contribuer à préciser ces critères en posant des questions au PO. Ces questions peuvent être posées lors des réunions de conversation, où le PO présente les US une par une, et l'équipe pose des questions pour connaître les détails. L'ensemble des points discutés lors de cette réunion devraient être capturés et alimenter les critères d'acceptation.
- Une fois ces critères formalisés, les testeurs pourront spécifier des tests d'acceptation spécifiques à chaque critère. Ils pourront par exemple prévoir des cas passants et non passants, des cas aux limites, etc. Le nombre et la profondeur des tests à prévoir doit être lié à la criticité ou au niveau de risque de la US associée.

- Quel est le meilleur moment pour les écrire ?

- Les critères d'acceptation font partie de la user story et, à ce titre, devraient être rédigés au moment où la user story est créée.
- Ils devraient dans tous les cas être disponibles avant que la user story soit développée par l'équipe, car ils fournissent des indications précieuses aux développeurs et aux testeurs.
- Il serait utile de les avoir avant même que la user story soit estimée par l'équipe, car ils peuvent fournir des indications sur la complexité de développement et de test.

- Techniques de rédaction

- Format checklist d'acceptation
 - Surtout à destination des PO et des testeurs pour définir l'ensemble des points à vérifier pour déclarer une US terminée (*Done*).
 - Chaque critère d'acceptation peut être priorisé.
- Format « Given / When / Then »
 - Formalisme utilisé dans la méthode BDD (*Behavior Driven Development*), surtout à destination des développeurs pour créer des cas de test d'acceptation (au niveau US) dans un langage compréhensible par les équipes métier.
 - S'appuie le plus souvent sur un framework de test (ex. *cucumber*) permettant d'automatiser ces tests pour les rejouer systématiquement à chaque commit de code.
- Format « Entrée / Résultat attendu »
 - Utilisé dans la méthode ATDD (*Acceptance Test Driven Development*) pour spécifier des valeurs d'entrée à tester.
 - S'appuie sur le framework FitNesse.



Il est préférable de choisir une seule technique adaptée au contexte et de l'appliquer systématiquement.
Cela évite les problèmes d'interprétation en fonction de l'équipe qui les rédige.

Rédaction

- Consignes à appliquer pour l'écriture des critères
 - Utiliser des phrases simples
 - Un critère d'acceptation n'est pas une règle de gestion.
 - Il doit permettre d'identifier rapidement le résultat Pass / Fail.
 - Si un critère est défini pour vérifier une règle de gestion complexe, les détails ne devraient pas apparaître dans l'intitulé ; il est préférable de fournir un renvoi vers la spécification associée.
 - Hiérarchiser et prioriser les critères
 - Définir un nombre maximum de critères par US : si ces critères sont trop nombreux, c'est souvent le signe que la US est trop complexe et doit être découpée davantage.
 - Commencer par les critères les plus évidents, en restant au plus près possible de l'objectif de la US, ex. « Après la validation du formulaire par l'utilisateur, l'application affiche l'écran suivant ».
 - Ajouter ensuite les critères d'acceptation secondaires : affichage de la page, actions / champ de saisie obligatoires et facultatifs, réponses attendues du système...
 - Ajouter les critères d'acceptation techniques, par exemple « le formulaire doit être sauvegardé dans la base de données client ».
 - Utiliser une nomenclature et une numérotation des critères d'acceptation en lien avec les US, ex. US_1_AC_n

Exemples de spécification de critères d'acceptation

- User story définie

En tant que **client de la banque**
Je veux retirer de l'argent au distributeur
Afin de ne pas faire la queue au guichet

- Exemples de critères d'acceptation

1. Cas nominal : un client réel avec compte créditeur et carte valide doit pouvoir retirer la somme demandée
2. Le système doit afficher des messages d'erreur pour les conditions :

- Client inexistant
- Compte débiteur
- Carte invalide

3. Le système doit gérer les cas d'interruption suivants :

- Annulation demandée par le client
- Rejet de la carte par la machine
- Temps de réponse du client supérieur à 1 minute

4. Le système doit gérer le montant de retrait maximum défini pour chaque type de carte supportée

2.8 Planification

Du management du projet traditionnel aux modèles agiles

Projet Agile



Analyse			
A	C	R	T
A	C	R	T
A	C	R	T
A	C	R	T
A	C	R	T
A	C	R	T
A	C	R	T
A	C	R	T
A	C	R	T
A	C	R	T

Sprint 0



Jalons

Backlog
Produit
Priorisé

Sprint 1



Incrément
validé

Sprint 2



Incrément
validé



Sprint N

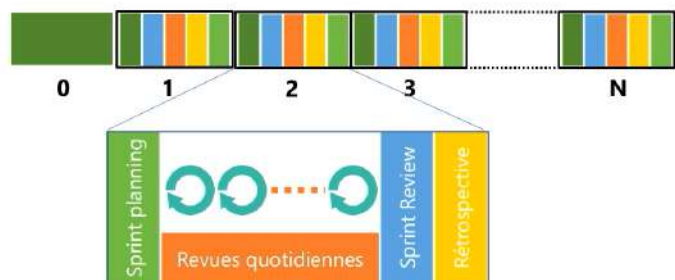


Produit
terminé

- Itérations courtes
- Logiciel opérationnel délivrant des éléments de valeur
- Parallélisme et chevauchements considérables des activités
- Activités de test tout au long de l'itération

Planification du SPRINT

- Le suivi du sprint est assuré par deux éléments
 - La Daily Scrum
 - Le ScrumBoard



- Le sprint se conclut par deux activités
 - Le Sprint Review
 - La Rétrospective

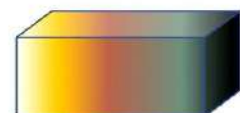
Jalons



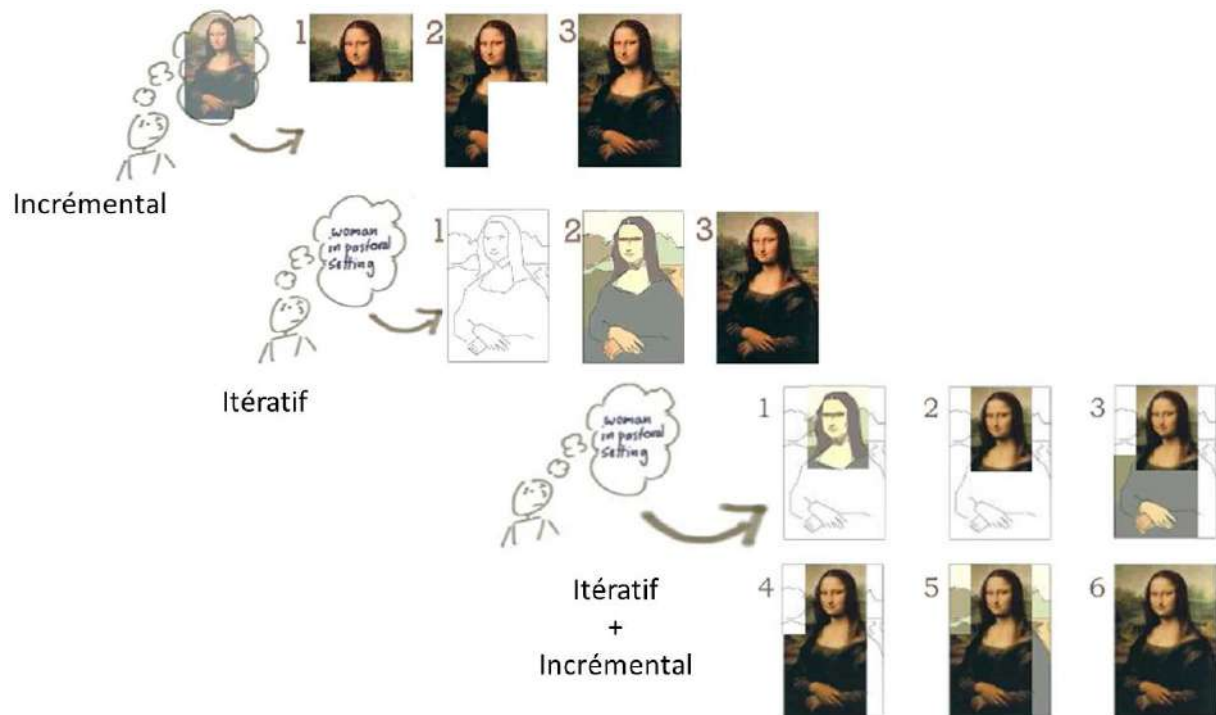
Itérations



Livraisons



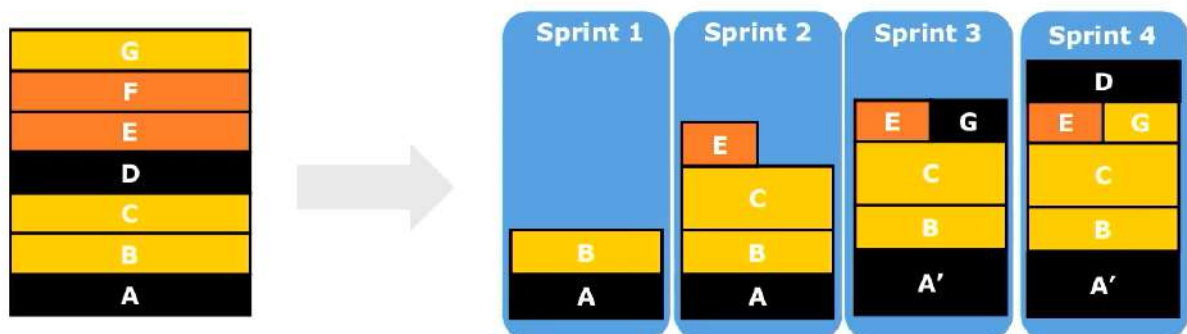
Itérations et incréments



2.9 Artefacts

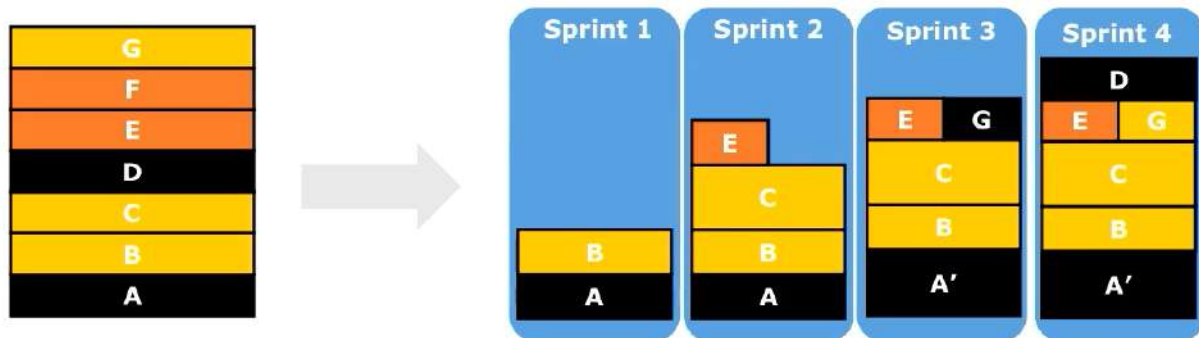
Le Backlog produit

- Défini par le Product Owner
- Liste priorisée et planifiée de fonctionnalités
- Peut bouger d'un sprint à l'autre



Le Backlog de sprint

- Choisi par l'équipe à chaque début de sprint
- Sélection faite sur le principe de traction (*pull*)
- En fonction de la capacité à faire (vélocité)



Les critères < Done > (terminé)

Team "Done" List

- ...With a Story
- All Code (Test and Mainline) Checked in
 - All Unit Tests Passing
 - All Acceptance Tests Identified, Written & Passing
 - Help File Auto Generated
 - Functional Tests Passing

- ...With a Sprint
- All Story Criteria, Plus...
- Product Backup Updated
 - Performance Testing
 - Package, Class & Architecture Diagrams Updated
 - All Bugs Closed or Postponed
 - Code Coverage for all Unit Tests at 80% +

Vision itération de développement

- ...Release to INT
- All Sprint Criteria, Plus...
- Installation Packages Created
 - MOM Packages Created
 - Operations Guide Updated
 - Troubleshooting Guides Updated
 - Disaster Recovery Plan Updated
 - All Test Suites Passing

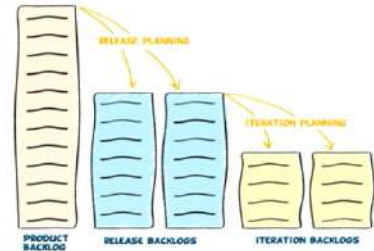
- ...Release to Prod
- All INT Criteria, Plus...
- Stress Testing
 - Performance Tuning
 - Network Diagram Updated
 - Security Pass Validated
 - Threat Modeling Pass Validated
 - Disaster Recovery Plan Tested

Vision installation des environnements

2.10 Release Planning

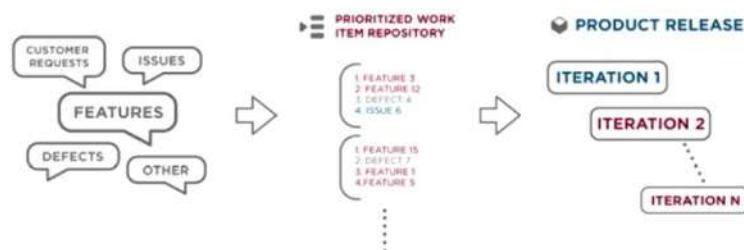
- Comme un projet classique, le projet agile a des contraintes :

- De délai
 - Mise en production d'un projet réglementaire
 - Mise sur le marché d'un logiciel éditeur (*time-to-market*)
- De périmètre
 - Besoin exprimé par le client
 - Fonctionnalités nécessaires
 - Impératifs concurrentiels



- Contrairement à un projet classique
 - Le logiciel est développé en plusieurs étapes (Release / Sprints)
 - Chaque étape fait l'objet d'une planification, d'un suivi et d'une validation

- Une release est un ensemble d'itérations



- D'un point de vue produit, elle correspond à un backlog cohérent de fonctionnalités.
- D'un point de vue logiciel, elle constitue une version majeure prévue pour une mise en production.

Planification de releases

- Réunion/itération effectuée avant le début des itérations
- La base de travail est le Backlog produit
- Le PO donne une estimation de priorité des User Stories
- L'équipe donne une première estimation de charge
- Tout le monde identifie les risques projet et qualité
- Le résultat est un planning
 - Des dates et du contenu de chaque release
 - Du contenu des premières itérations

L'estimation de l'effort

- Pourquoi ?
 - Déterminer l'effort de test nécessaire dans la release
 - Compléter l'estimation d'effort de l'équipe
- Quand faut-il la réaliser ?
 - Lors du release planning (sprint 0)
 - Quand le PO le demande
- Qui participe ?
 - Toute l'équipe doit participer avec PO et Scrum Master
- Comment ?
 - Planning Poker

Le Panning Poker

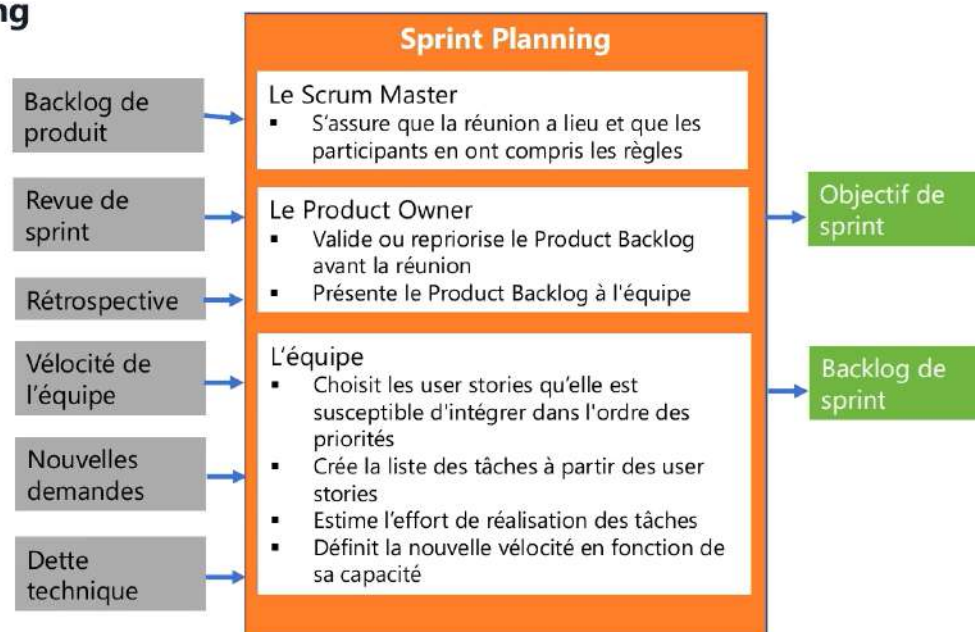
- Méthode d'estimation relative basée sur le consensus
- Un jeu de cartes identique distribué à chaque équipier
- Fonctionnement
 - Chaque Story est lue aux évaluateurs par le Product Owner
 - Chacun discute et pose des questions au Product Owner
 - Chaque équipier pose sa carte
- Chaque carte a une valeur différente
 - Numérique (0 à 100) représentant
 - Les points de Story
 - Les jours d'effort
 - Cartes spéciales
- Analyse du résultat
 - Si toutes les estimations convergent, la valeur est attribuée
 - Sinon les discussions reprennent jusqu'à aboutir à un consensus



2.11 Sprint Planning

- Le Sprint Planning permet de définir le Sprint Backlog
 - Sous ensemble du Product Backlog
 - Correspondant à un accord mutuel entre le Product Owner et l'équipe
 - Le Product Owner ayant toujours le dernier mot
- Le Sprint Planning se découpe en deux étapes
 - Définition du contenu du Sprint (quoi ?)
 - Estimation et découpage en tâches du contenu du Sprint (comment ?)

Sprint planning



L'objectif du sprint

- Il est un élément indispensable à atteindre pour que l'on puisse considérer que le sprint est une réussite.
- C'est un énoncé bref de ce sur quoi l'équipe va travailler pendant le sprint.
 - « Mise en œuvre du système de réservation en ligne des croisières ».
 - « Réalisation des échanges entre le site web et l'ERP concernant les réservations en lignes ».
- Il permet à tout le monde de comprendre l'incrément qui sera livré.
 - Définition d'une fonctionnalité cohérente.
 - Aménagements techniques sur l'architecture de l'application.
- Si des arbitrages doivent avoir lieu durant le sprint, c'est souvent l'objectif du sprint qui permettra de faire pencher la balance.

Sprint Backlog

- A partir des éléments sélectionnés dans le product backlog, l'équipe détermine comment elle va construire un incrément à partir du backlog.
- Elle définit le plan de travail pour chaque user story.
 - Décomposition en tâches de développement, documentation et test
 - Chaque tâche devrait durer quelques jours au maximum afin de pouvoir être suivie convenablement
- Elle réestime la charge de travail pour chaque tâche.
- Elle clarifie avec le PO le contenu de chaque tâche, notamment pour la documentation et les tests.
- Elle peut proposer une révision du backlog si elle considère qu'elle sera trop ou trop peu chargée pendant le sprint.
- Elle s'engage sur un périmètre de sprint – le sprint backlog.

ID	Title	Story Points	Assigned To	Priority
513	Retour ergonomie générale	2	Tom	1
511	Charger des contrats et devis factices pour cas de test	7	John	1
263	Enregistrement tarification	13	Mary	1
506	Retours ergonomie véhicule	2	Bob	1
504	[Bug] Ergonomie champs recherche avancée	1	John	1
505	Fusionner le menu souscription et tarification	3	Lisa	1
262	Garanties et tarifs procédure stockée	4	Tom	1
484	Indexation GED	5	Bob	1
261	Conducteur(s) et antécédents (gestion des permis)	3	Mary	1
516	Appel Ged et GRC	1	Mary	2
515	Gestion des Modals	2	Mary	2
512	Profiler les APIs	2	Lisa	2
508	Reprise Brouillon	5	John	2
509	Duplication de devis	9	Lisa	2
498	Prise en compte des retours	5	Tom	2
514	Reprise d'un devis par l'agent accepte ou refuse	4	John	2
497	Reprise de devis par gestion	4	John	2
289	Ecrans souscription	5	Bob	2
493	Créer une action pour l'extérieur pour ouvrir une consultation contrat/devis	1	Lisa	3

Les règles

- On n'assigne jamais une user story à l'équipe.
- L'équipe définit son propre potentiel de production (vélocité).
- Elle s'auto-organise pour l'affectation des tâches.
- L'équipe peut proposer et justifier un ordonnancement du product backlog différent.
- Le product owner aura toujours le dernier mot.
- On peut dans cette phase estimer les user stories qui ne le sont pas encore.
- On va travailler généralement sur 60/70 % du product backlog.

2.12 Suivi du Sprint

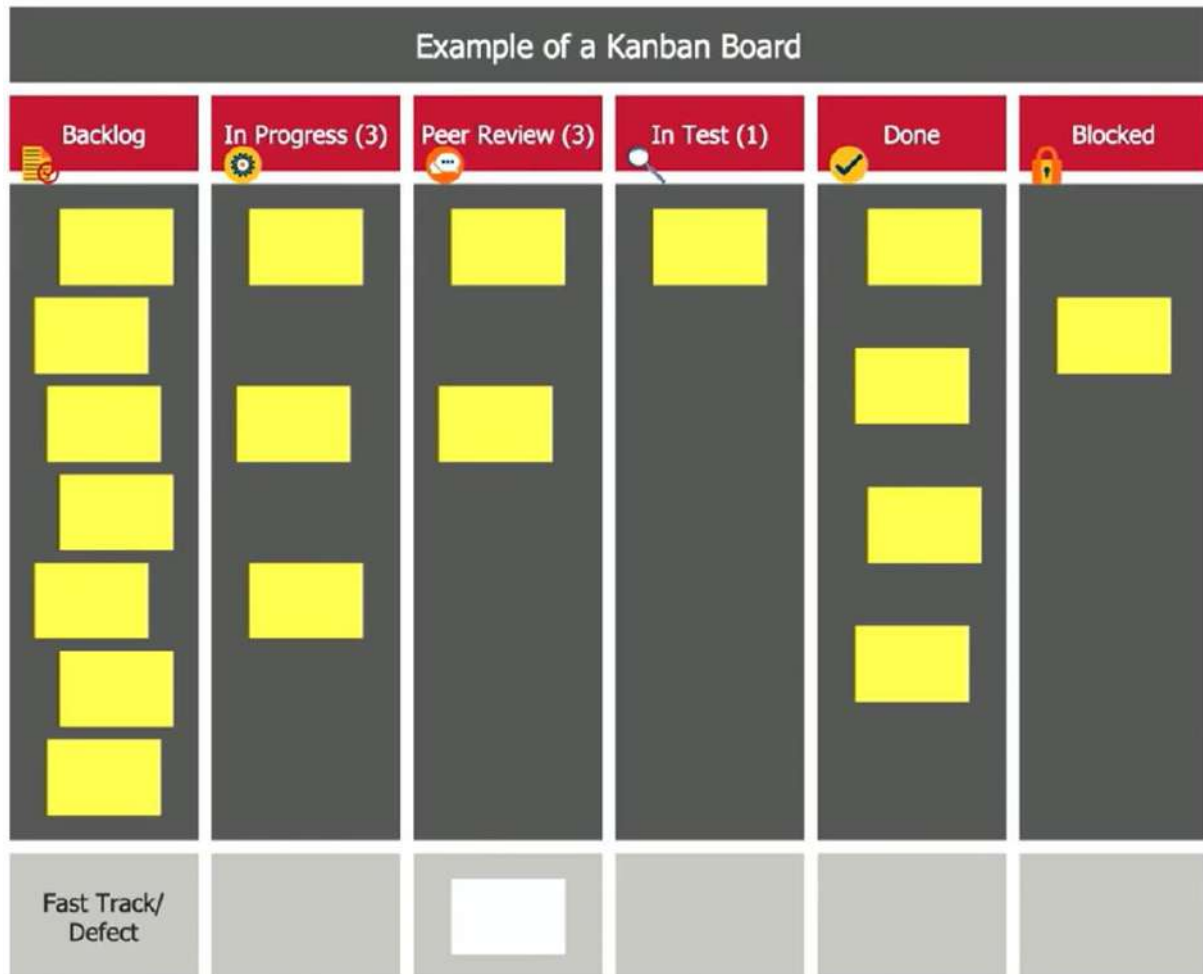
- La planification des tests sur une itération
 - Quel est le périmètre de test ?
 - Définir ce qui est testé et ce qui ne l'est pas
 - Quels sont les objectifs de test sur l'itération ?
 - Tests de support à l'équipe ou orientés client
 - Quels sont les risques ?
 - Couverture attendue, profondeur des tests et priorisation
 - Qui sont les testeurs ?
 - La qualité est la responsabilité de tous
 - Quels sont les environnements et données de tests nécessaires ?
 - Existe-t-il des prérequis et des dépendances pour les tests ?
 - Ordonnancement des tests dans l'itération

Le Daily Scrum (ou standup)

- Cérémonie quotidienne et minutée
 - Communication au sein de l'équipe
 - Visibilité sur l'ensemble des travaux
 - Identification précoce et résolution collective des blocages
- 3 questions pour chacun
 - Qu'as-tu fait hier ?
 - Qu'es-tu en train de faire ?
 - Quels sont les blocages ?
- Mise à jour du tableau de bord
 - Visualisation de l'avancement du projet
 - Mesure de l'écart par rapport aux estimations



Kanban Board



Exemple Kanban pour un Testeur Logiciel

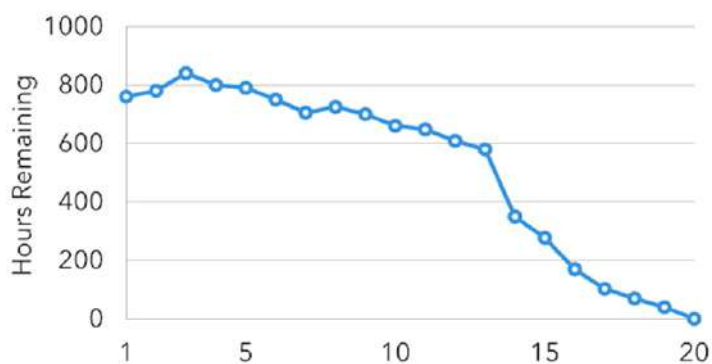
Backlog	À faire	En cours	En test	Done
US-101	Cas de test Login	Automatisation Selenium	Validation QA	✓
Bug-45	Tests API	JMeter	Régression	✓
US-102	Accessibilité	BDD Cucumber	UAT	✓

Le Daily Scrum- les règles

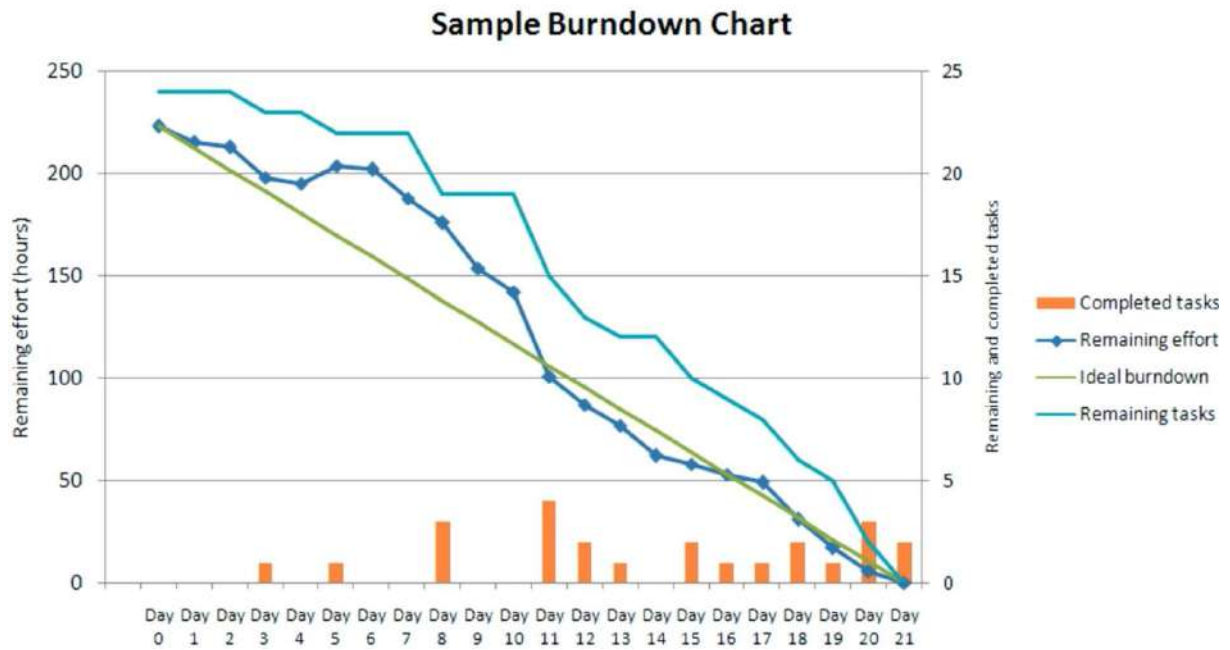
- C'est l'outil de l'équipe
 - Le Scrum Master et le Product Owner sont optionnels.
 - S'il participe, le Scrum Master doit se faire discret lors du stand up et cependant s'assurer que le Daily Scrum se déroule comme convenu.
 - Le Product Owner peut profiter de ce moment pour suivre l'avancement du projet.
 - Les parties prenantes peuvent assister mais ne doivent pas intervenir.
- Seuls les membres de l'équipe peuvent parler
 - Chacun doit s'exprimer brièvement car le Daily Scrum est *timeboxé* à 15 minutes.
 - Ne pas dévier sur des conversations annexes (règlement d'un point technique complexe, modification des priorités, etc.), ni tenter de régler des problèmes plus sérieux. Il s'agit de les repérer pour les régler ultérieurement.
 - Ce n'est pas non plus le lieu des règlements de compte (l'équipe doit comprendre qu'elle doit s'unir pour réussir).
 - Chacun s'engage car l'équipe est responsable.
 - On ne fait pas de compte rendu au Product Owner ou au Scrum Master.
- Il devrait se dérouler systématiquement à la même heure et dans le même lieu
 - Le Daily Scrum se déroule impérativement debout.
 - Il améliore la communication et permet d'éviter d'autres réunions.
 - Il faut trouver un lieu tranquille mais proche des « radiateurs d'information ».

Burndown Chart

- Suit la réalisation des tâches au quotidien à partir des informations communiquées par l'équipe
- Mis à jour par le Scrum Master
- Permet d'affiner le périmètre du sprint

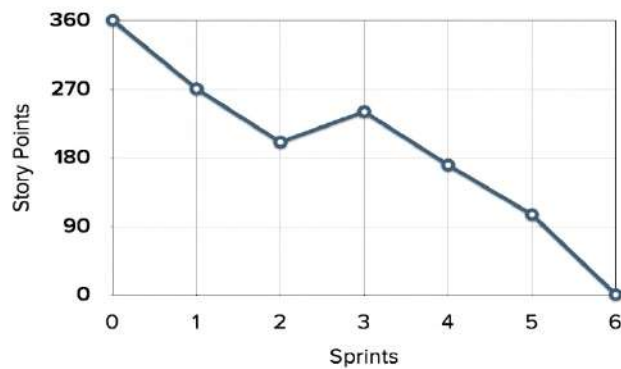


<http://www.mountangoatsoftware.com/>



Burndown chart de la release

- Suit l'avancement d'une release par rapport à ce qui a été planifié
- Mis à jour par le Scrum Master à la fin de chaque sprint à partir de tous les éléments « done »
- Permet de maintenir le projet agile sur les rails



<http://www.mountaingoatsoftware.com/>

- Gérer les adaptations du plan
 - Facteurs internes
 - Modification du besoin initial
 - Problèmes techniques
 - Vitesse de l'équipe
 - Sous-estimation de la complexité
 - Facteurs externes
 - Pression commerciale, opportunités...
 - Changement d'objectifs du projet
- Impact inévitable sur l'activité du projet



- Adapter le backlog et le planning des sprints
- Garder l'image globale de la release
- Anticiper suffisamment sur la conception

La revue de sprint

- En fin de chaque itération
- L'équipe effectue la démonstration de ce qui a été produit au PO
- L'équipe et le PO passent en revue les objectifs énoncés lors du sprint planning
- Le PO valide le sprint si l'objectif général est atteint
- L'équipe mesure sa vitesse réelle à partir des activités réalisées

Calcul de la vitesse de l'équipe

- C'est la somme des points des user stories qui ont été terminées lors du sprint écoulé

Utilité :

- Estimer le nombre de points que l'équipe est capable de réaliser avant de planifier un sprint
- Indice de performance de l'équipe
- Estimer la durée du projet



2.13 Rétrospective



- La rétrospective
 - Réunion collective en fin de chaque itération
 - Concerne surtout l'équipe, avec le SM et éventuellement le PO
 - Dans un environnement professionnel de confiance mutuelle
 - Utilisée pour faire le point sur
 - Ce qui est réussi
 - Ce qui peut être amélioré
 - Comment intégrer les améliorations
- Domaines concernés par la rétrospective
 - Organisation, processus et outils
 - Personnes et aspects relationnels dans l'équipe
- Peut aboutir à des décisions
 - Fonctionnement et cohésion d'équipe
 - Propositions d'amélioration

Facteurs clés de succès pour les rétrospectives

- Effectuer des analyses de causes racine
 - Réfléchir aux problèmes avant d'apporter des solutions
- Implémenter seulement quelques améliorations par itération
 - Eviter de tout réinventer à chaque itération
 - Maintenir un rythme continu
- Impliquer les bonnes personnes
 - Équipe
 - Représentants métier
 - Testeurs
- Conduite de la réunion : cf. revues

Exemple – rétrospective hebdomadaire sprint n° 03

- Ce qui a été réussi
 - Atteinte de l'objectif du sprint : implémentation des filtres de recherche premier niveau
 - Feed-back efficace de la qualité de la release
- Ce qui pourrait être amélioré
 - Problème de disponibilité du représentant métier pendant ses congés
 - Proportion importante de tests non automatisés → risque !
- S'engager sur les améliorations
 - Désigner un back-up métier capable de reprendre la fonction du représentant métier pendant les absences de ce dernier
 - Renforcer l'automatisation des tests

K: Ce que l'on garde

L'indicateur sur le suivi du chiffre d'affaires

L'analyse mensuelle des résultats

I: Ce que l'on améliore

La remontée automatique du résultat du KPI n°3 dans le TDB

La mise en forme du KPI n° 4 (pas assez « clignotant »)

S: Ce que l'on commence

Un indicateur prédictif lié à notre présence sur les réseaux sociaux

S: Ce que l'on arrête

L'indicateur sur le taux de transformation des devis (pas exploitable)

L'affichage des résultats sans explication aux équipes

neo facto



Mettre à dispo. un glossaire

Plus de TP "distanciel"

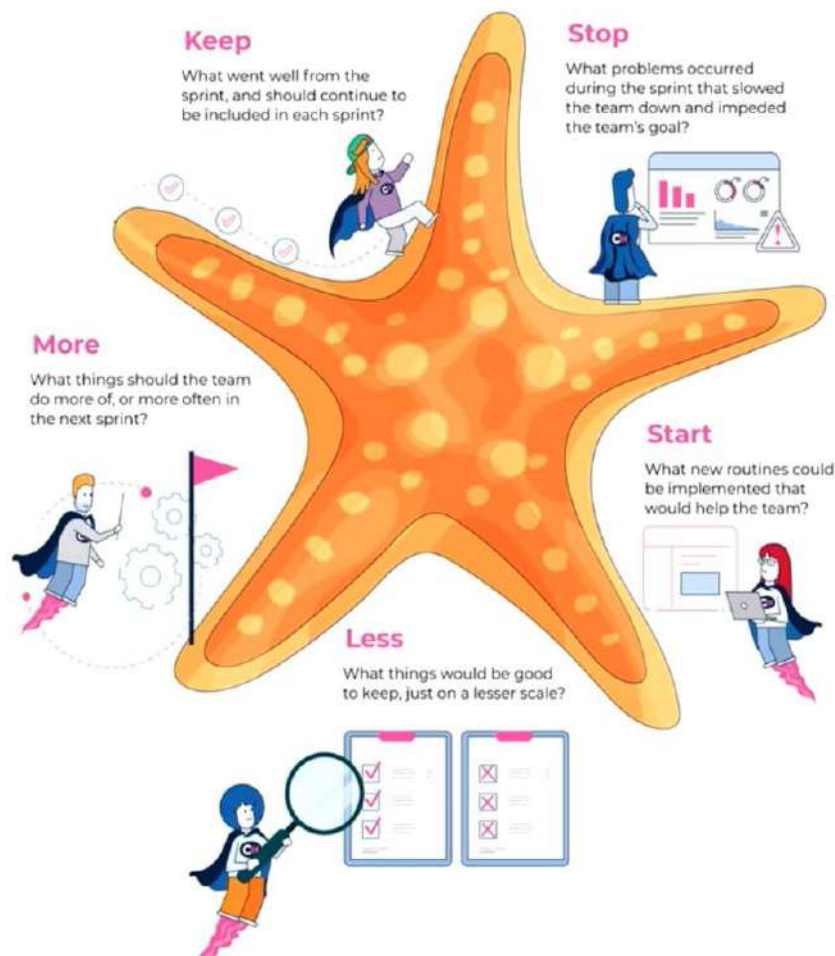
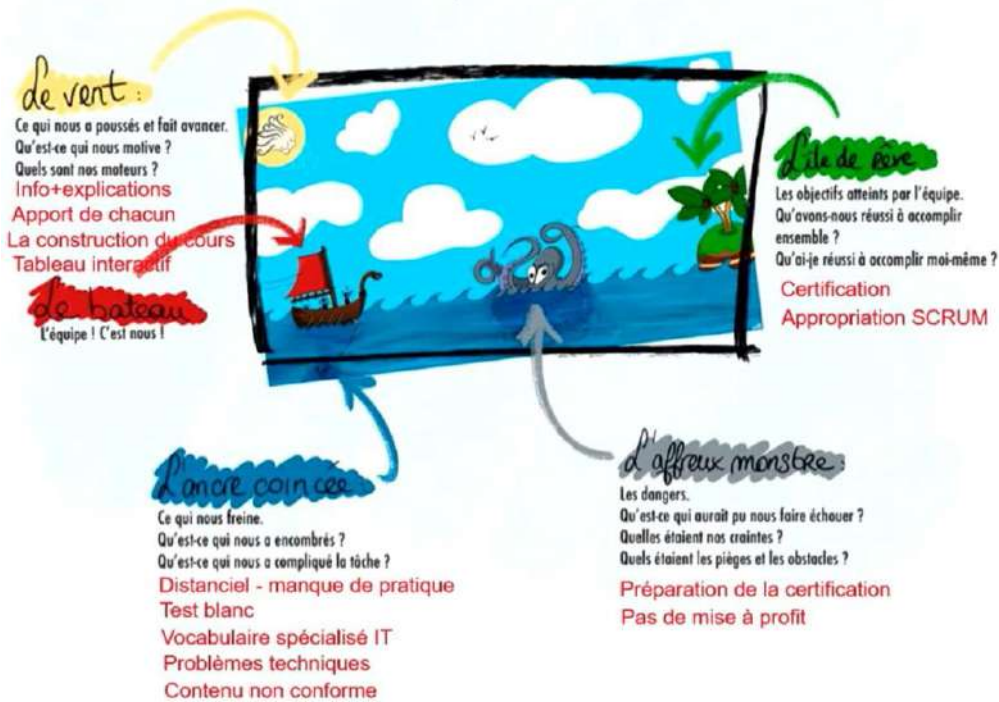
"Pratique + test blanc (certification)"

Revoir le contenu (Cf. prérequis)

de Speedboat

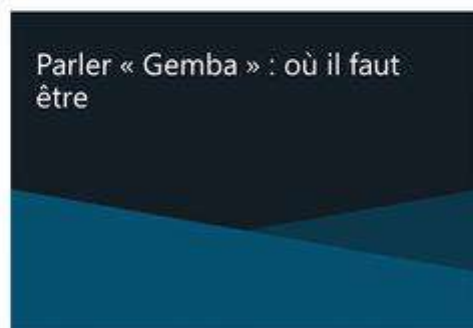
Par Linda Simon

Documents disponibles



Module 3- Conduite du changement

3.1 Objectifs du module



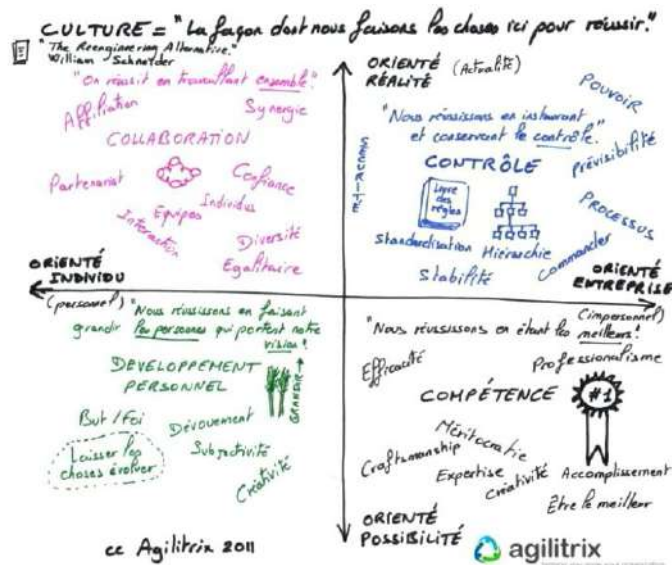
3.2 Culture

La rupture culturelle

Appréhender la culture existante par le modèle de Schneider

- **Collaboration**
 - Capacité à travailler ensemble
- **Contrôle**
 - Avoir et garder le contrôle
- **Compétence**
 - Être le meilleur
- **Développement personnel**
 - Apprendre et se développer en visant un objectif ayant du sens

Le modèle de Schneider



Axe horizontal
 Orienté individu (personnel) versus
 orienté entreprise (impersonnel)

Axe vertical
 Orienté réalité (actualité) versus
 orienté possibilité

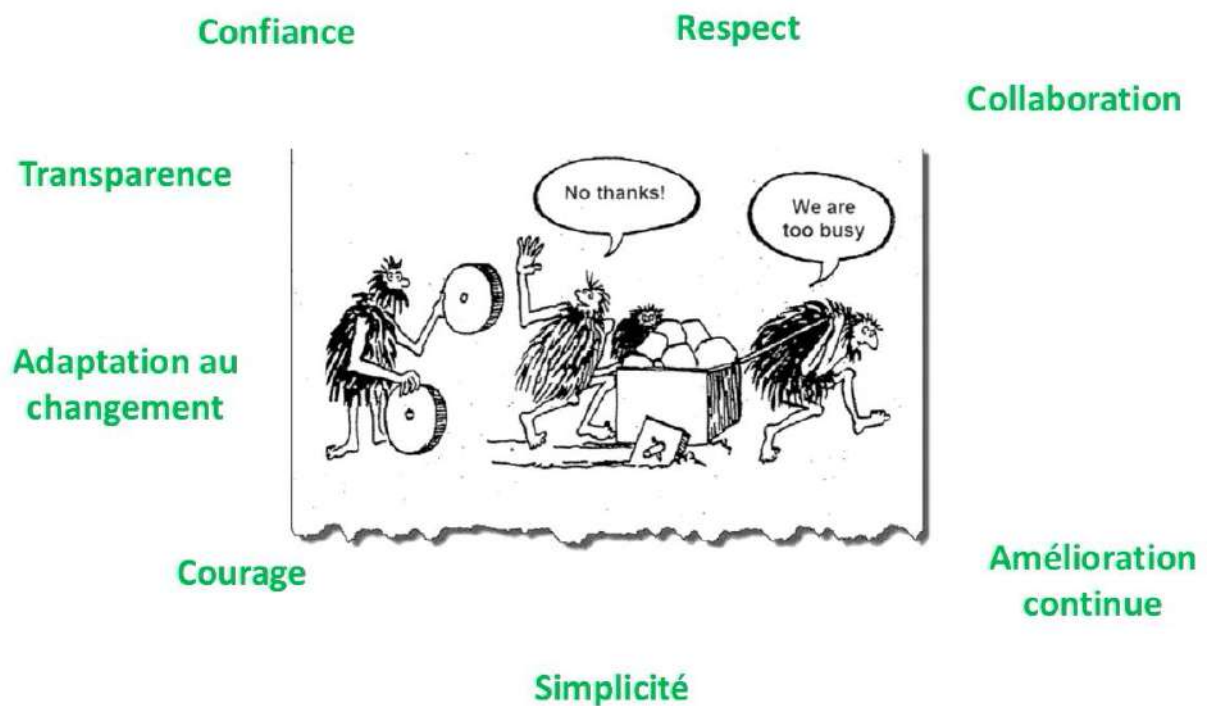
Image issue du site <https://sjaillais.wordpress.com/tag/william-schneider/>

Les questions posées par l'agilité

- Quelle est la culture de mon entreprise en ce moment ?
- Cette culture est-elle en phase avec celle de l'agilité ?
- À quels problèmes dois-je m'attendre si les deux cultures ne sont pas en phase ?
- Serons-nous capables d'« être agiles » ou de « faire agile » ?



Les valeurs fondamentales



- La confiance
 - Excellent levier de motivation intrinsèque
 - Confiance entre
 - Un Manager et ses collaborateurs
 - Co-équipiers
 - Un client et son partenaire
 - Sans confiance, pas d'équipe...



- La collaboration
 - Avec le client
 - Plus fréquemment
 - En confiance
 - Engagement mutuel
 - Dans l'équipe
 - De manière constante
- La transparence
 - Pour identifier et lever les blocages
 - Pour être plus efficace
 - Pour s'entraider, collaborer
 - Pour s'adapter
 - Pas pour contrôler !!



© Scott Adams, Inc./Dist. by UFS, Inc.

- La simplicité
 - C'est arrêter de dire « *Qui peut le plus peut le moins* »
 - C'est se limiter à ce qui est réellement utile
 - « *Juste ce qu'il faut, juste à temps* »

- L'adaptation au changement
 - Avoir un objectif clair
 - Une idée du (des) chemin(s) à prendre
 - Être prêt à changer en cours de route

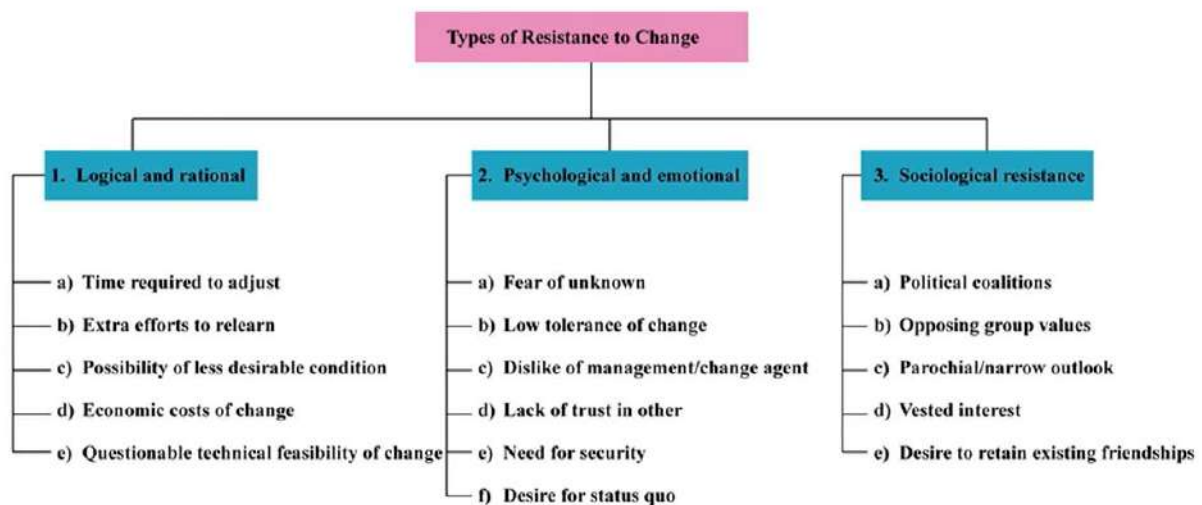


- L'amélioration continue
 - Au cœur des approches empiriques
 - L'agilité fournit un cadre, mais pas la recette
 - Adapter les pratiques au contexte en fonction
 - Du produit développé
 - Des talents et compétences de chacun
 - De la culture d'entreprise
 - En maintenant l'adhésion à des règles communes
 - S'appuie sur les *personnes qui font* (les « faiseurs »)
- Le courage
 - Être transparent demande du courage
 - Changer, s'adapter demande du courage
 - Rester simple demande du courage
 - Accepter de se tromper demande du courage
 - Se remettre en question demande du courage
 - Etc.

- Le respect
 - Préalable à toute communication
 - Et donc à toute collaboration
 - Respect des personnes
 - Respect de soi-même
 - Respect des autres co-équipiers

3.3 Conduite du changement

- En introduisant Scrum dans votre société, vous allez :
 - Provoquer le changement
 - Bousculer les habitudes
 - Être attentif aux facteurs bloquants



Il existe six façons d'anticiper et assouplir la résistance au changement (Kotter et Schlesinger) :

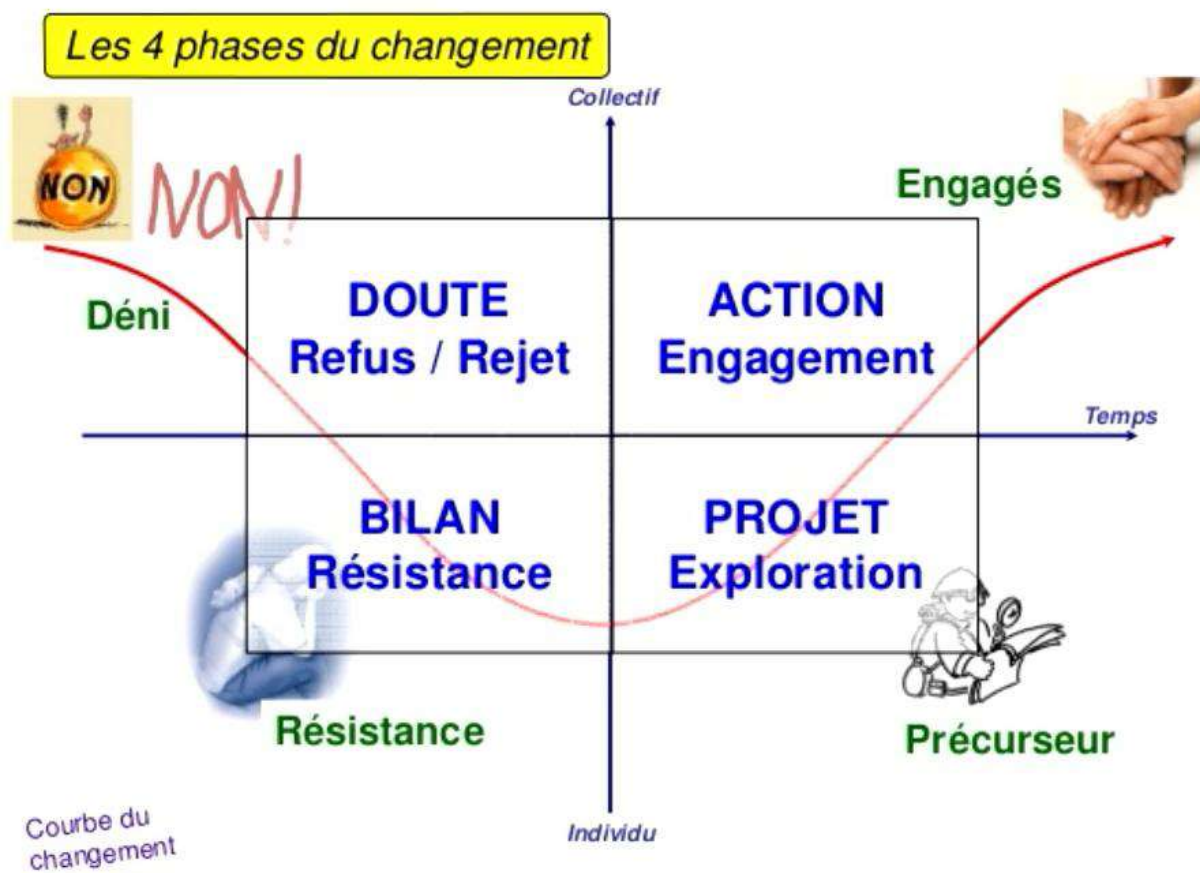
1. Éducation et communication
2. Participation et implication
3. Facilitation et support
4. Insertion (et manipulation)
5. Négociation
6. Coercition explicite et implicite

3.4 Gemba

Gemba : du japonais « le lieu réel »

現場

Conduite du changement



Objectif du Gemba

Le **Gemba** (Genba) a pour but d'aider dans les prises de décision ou dans l'amélioration. Cela peut paraître bête à dire mais on ne sait jamais ce qu'il se passe quand on n'est pas directement sur le terrain ; penser savoir ou imaginer ce qu'il se passe sont des comportements classiques mais qui mettent souvent les gens dans des positions délicates : ils sont dans l'erreur sans même le savoir.

Dans le **Lean Management**, avec le Gemba, nous allons alors inciter les managers à aller directement sur le terrain pour prendre de bonnes décisions, pour comprendre les améliorations à réaliser.

A graphic of a clipboard with a red clip at the top. The clipboard has an orange background with a blue border. The title 'Gemba Walk Checklist' is written in white. To the right of the checklist is a yellow pencil. The checklist contains seven items, each with a green checkmark icon to its left.

Gemba Walk Checklist

- ☒ What are you currently working on?
- ☒ Is there an established process for this type of work?
- ☒ Do you have any problems with the established process?
- ☒ Why is there a problem?
- ☒ How can you fix the problem?
- ☒ What do you do to recognize the root cause of the problem?
- ☒ Who do you speak with if there is a certain problem?

La marche du Gemba



Aller et voir. L'idée de base de la marche Gemba est que les responsables et les meneurs de tous les niveaux doivent régulièrement passer dans l'atelier et s'impliquer dans la recherche de gaspillage.



Demander pourquoi : le principal objectif d'une marche Gemba est d'examiner la chaîne de valeur en détail et d'identifier les parties problématiques grâce à une communication active. Le bon meneur est toujours davantage disposé à écouter plutôt qu'à parler.



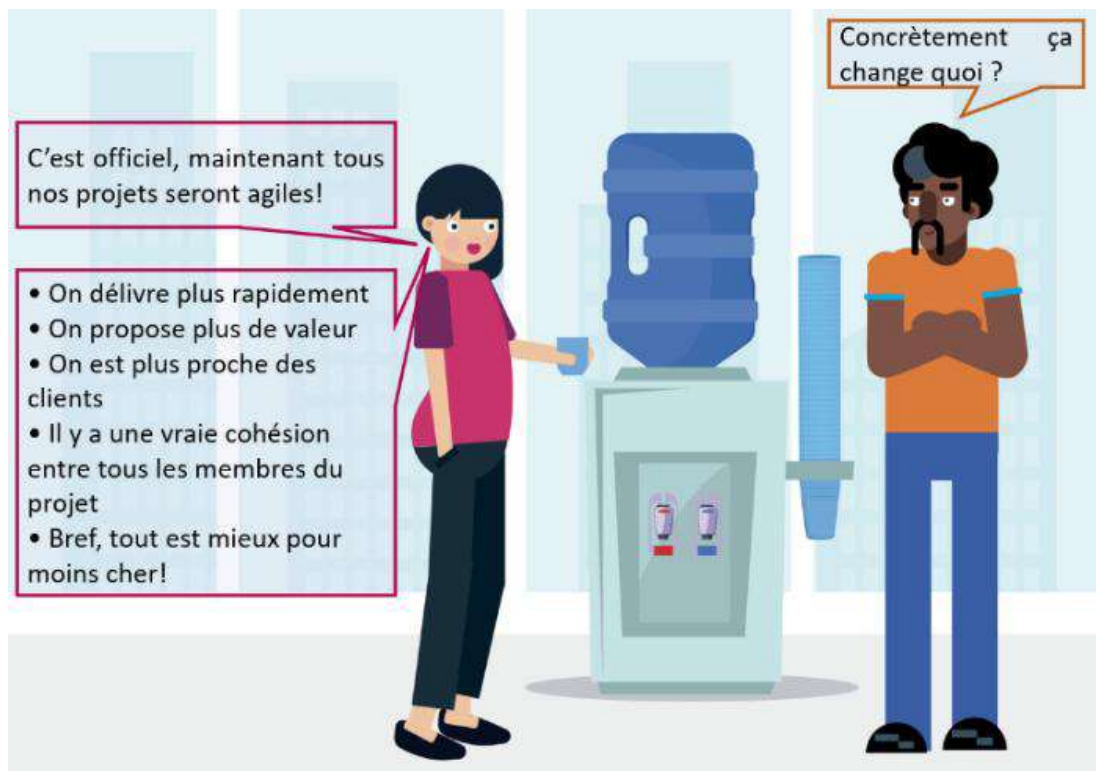
Respecter les individus. Gardez en tête qu'une marche Gemba n'est pas une « marche de chef ». Porter des accusations et blâmer les individus n'est pas ce que vous devez faire. Vous n'êtes pas là pour juger et évaluer les résultats. Vous êtes là pour collaborer avec l'équipe et trouver les problèmes ensemble. Essayez de vous concentrer sur le fait de trouver les points faibles du processus, pas les individus.

3.5 Références pour aller plus loin

Pour aller plus loin

- **The Scrum Alliance** : www.scrumalliance.org
- **Agile Alliance** : www.agilealliance.org
- **Mike Cohn** : www.mountaingoatsoftware.com
- *Agile project management with Scrum*, Ken Schwaber
- *Lean Software Development*, Mary & Tom Poppendieck
- *The Enterprise and Scrum*, Ken Schwaber
- *Agile Estimating And Planning*, Mike Cohn
- *User Stories Applied: For Agile Software Development*, Mike Cohn
- *Agile Estimating And Planning*, Mike Cohn
- *Writing Effective Use Cases*, Alistair Cockburn
- *Agile Retrospective, Making Good Teams Great*, Esther Derby et Diana Larsen, Pragmatic Bookself
- *Project Retrospectives, A Handbook for team reviews*, Norman L. Kerth – ed. Dorset House Publishing

Atelier 1 : Le testeur à la découverte du monde Agile



L'objectif de l'atelier sera de :

- Délivrer l'ensemble des US pour la fin du sprint
- Planifier les tâches selon les compétences

Contraintes

- Equipe constante
- Tâches variées
- Piques d'activités
- Livraison d'une user story = finir toutes les tâches liées

Accéder à l'activité suivante, et prenez en charge ces tâches

- Planifier les activités en fonction des compétences de l'équipe
- Planifier les activités en fonction d'un ordre spécifique

Codes couleurs

- Les tâches en bleu foncé sont les développements complexes.
- Les tâches en bleu clair sont les développements simples.
- Les tâches en orange foncé sont les spécifications complexes.
- Les tâches en orange clair sont les spécifications simples.
- Les tâches en vert foncé sont les tests complexes.
- Les tâches en vert clair sont les tests simples.

Atelier 1

Composition US du sprint – Drag & Drop

Gleissez-déposez les rectangles (une case) depuis la palette vers la grille. Complexe = 3 cases, Simple = 2 cases. Chaque use case ne peut être utilisée qu'une seule fois.

Exporter JSON Réinitialiser

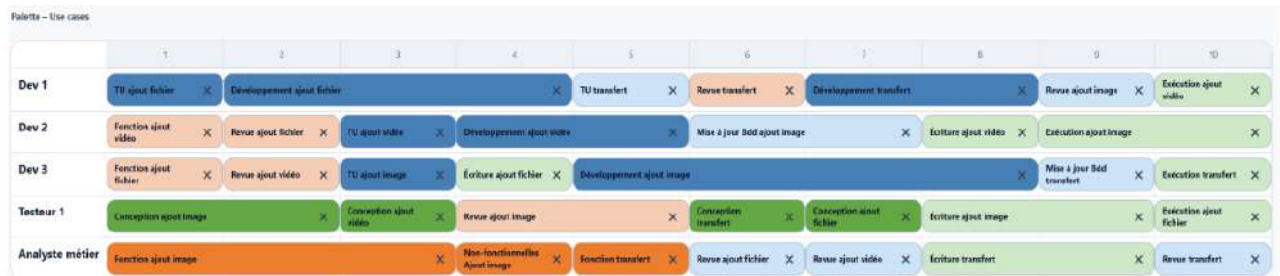
Palette – Use cases

Fonction ajout image largeur: 3	Revue ajout image largeur: 2	Non-fonctionnelles Ajout image largeur: 1	Développement ajout image largeur: 4	TU ajout image largeur: 1	Mise à jour Bdd ajout image largeur: 2
Revue ajout image largeur: 1	Conception ajout image largeur: 2	Exécution ajout image largeur: 2	Écriture ajout image largeur: 2	Fonction ajout vidéo largeur: 1	Revue ajout vidéo largeur: 1
Développement ajout vidéo largeur: 2	TU ajout vidéo largeur: 1	Revue ajout vidéo largeur: 1	Conception ajout vidéo largeur: 1	Exécution ajout vidéo largeur: 1	Écriture ajout vidéo largeur: 1
Fonction ajout fichier largeur: 1	Revue ajout fichier largeur: 1	Développement ajout fichier largeur: 3	TU ajout fichier largeur: 1	Revue ajout fichier largeur: 1	Conception ajout fichier largeur: 1
Exécution ajout fichier largeur: 1	Écriture ajout fichier largeur: 1	Fonction transfert largeur: 1	Revue transfert largeur: 1	Développement transfert largeur: 2	TU transfert largeur: 1
Revue transfert largeur: 1	Mise à jour Bdd transfert largeur: 1	Conception transfert largeur: 1	Exécution transfert largeur: 1	Écriture transfert largeur: 1	

	1	2	3	4	5	6	7	8	9	10
Dev 1										
Dev 2										
Dev 3										
Testeur 1										
Analyste métier										

Correction Atelier 1

Il y a bien sur plusieurs possibilités de réponses à cet atelier, voici une possibilité d'organisation :



En Agile il est impossible de faire uniquement sa spécialité : Les équipes s'autoorganisent

- Il y a toujours des pics et creux de charge
- Il faut faire ce pour quoi on est le plus utile à l'équipe

C'est l'équipe Agile qui est responsable du produit et de ce qu'elle livre.

L'équipe est un collectif

La spécialisation reste aux spécialistes

- Certaines tâches ne peuvent être faites qu'à partir d'un certain niveau de compétence
- On ne demande pas à un testeur de développer aussi bien qu'un développeur (et vice versa)

Atelier 2- La qualité c'est pour tout le monde

Objectifs :

- Ecrire des spécifications
- Comprendre le suivi strict des spécifications
- Réaliser le bon produit en suivant strictement les spécifications

Contraintes

La qualité d'un produit dépend

- Des spécifications
- Des développements
- Des tests...

Lors de la création d'un mode d'emploi de construction du produit,

- La liste des actions doit être très précises et exhaustives
- Vous devez tout expliciter
- Si vous êtes en équipe, n'hésitez pas à
 - Exécuter strictement le mode d'emploi de l'autre équipe,
 - Analyser le résultat.

En équipe, La communication est essentielle !

- Pour limiter les erreurs
- Pour assurer une compréhension commune

Tous les acteurs participent à la qualité

- Chacun apporte sa pierre à l'édifice
- Impossible d'écrire des spécifications parfaites
- La bonne qualité résulte d'un effort collectif

La qualité va dépendre du contexte

- Notamment du public cible

Ecrivez dans un fichier un mode d'emploi de construction du produit :

- Préparer un verre de grenadine

Atelier 2 : Correction

Testeur Agile

Atelier 2

Proposition de solution

Correction de référence – Mode d'emploi pour préparer un verre de grenadine

Objectif

Obtenir un verre de boisson à la grenadine correctement dosé, homogène, prêt à être consommé.

Matériel nécessaire

- 1 verre cylindrique en verre transparent, capacité **250 mL** minimum.
- 1 cuillère à soupe (standard de **15 mL**).
- 1 carafe ou une bouteille d'eau potable remplie.
- 1 bouteille de sirop de grenadine.

Ingrédients

- **20 mL de sirop de grenadine** (équivalent à environ 1 cuillère à soupe bien remplie + 1/3).
- **180 mL d'eau fraîche** (température comprise entre 6 °C et 12 °C).

Étapes de préparation

1. Préparation du verre

- Poser le verre vide sur une surface plane et stable (exemple : une table).
- Vérifier que le verre est propre, sec, et ne présente aucune fissure.

2. Dosage du sirop

- Ouvrir la bouteille de sirop de grenadine.
- Verser dans la cuillère à soupe de la grenadine issue de la bouteille et



prélever **exactement 20 mL de sirop**.

- Si vous utilisez une cuillère de 15 mL, compléter avec environ **1/3 de cuillère supplémentaire**.
- Verser lentement le sirop au fond du verre en évitant les éclaboussures.

3. Ajout de l'eau

- Prendre la carafe ou la bouteille d'eau fraîche.
- Verser **exactement 180 mL d'eau** dans le verre contenant le sirop.
 - Si vous ne disposez pas d'un verre doseur, compter environ **12 cuillères à soupe d'eau**.
- Verser doucement pour éviter de faire mousser le mélange.

4. Mélange

- Introduire la cuillère propre dans le verre.
- Remuer le liquide avec un mouvement circulaire régulier pendant **10 secondes**.
- Vérifier visuellement que la couleur est homogène (rouge uniforme, sans zones plus foncées au fond).

5. Finition et vérification

- Retirer la cuillère et la poser sur un support adapté.
- Observer le niveau du liquide dans le verre : il doit être proche de **200 mL**.
- Goûter une petite gorgée : la saveur doit être sucrée et fruitée, ni trop concentrée, ni trop fade.

Résultat attendu

Un verre de grenadine :

- couleur rouge homogène,
- goût équilibré (ni trop sucré ni trop dilué),
- température fraîche agréable à la consommation,
- volume total autour de 200 mL.



Atelier 3 : Multiplier les exécutions

Objectifs :

- Contrôler le temps d'exécution des tests
- Conserver un bon niveau de qualité
- Continuer à développer son produit

Contraintes

- Construction itérative du produit
- Multiplication des exécutions des tests de régression
- Nombre de tests de régression croissant
- Taille de l'équipe constante

Pour cet atelier, à partir de l'application web fournie, il faudra retrouver 100 fois la bonne image parmi celles proposées.



Atelier 3

Résultat de l'activité



Atelier 3 : Bilan

On automatise instinctivement

- Toute tâche fastidieuse et répétitive
- Pour éviter l'ennui

L'automatisation est essentielle

- L'automatisation doit libérer du temps qui peut :
- Être affecté à d'autres tâches à plus forte plus-value
- Être utilisé aller plus vite

Le besoin d'automatisation est très important en Agile

- Construction incrémentale d'un produit
- Vérifications à chaque nouvel incrément

Atelier 4 : Importance de la communication

Objectifs :

- Assurer une compréhension commune de la fonctionnalité (et du produit) à l'ensemble de l'équipe
- Délivrer le produit souhaité par le client

Contraintes

- Nombreuses ambiguïtés
- Ce qui est évident pour l'un ne l'est pas pour un autre
- Des non-dits

Pour cet atelier, à partir de l'application web fournie, il faudra retrouver 100 fois la bonne image parmi celles proposées.



Répartition des rôles

Répartition des tâches par rôles

Associe chaque tâche au rôle qui en est principalement responsable (drag & drop des étiquettes dans les blancs).

- Le **Product Owner** ✓ priorise et maintient le backlog produit.
- Le **Scrum Master** ✓ anime les cérémonies et aide à lever les obstacles.
- Le/La **UX/UI Designer** ✓ conçoit les maquettes, prototypes et parcours utilisateur.
- Le **Développeur back-end** ✓ implémente les API et la logique métier côté serveur.
- Le **Développeur front...** ✓ intègre les écrans, composants et interactions côté client.
- Le **DevOps** ✓ automatise l'intégration et le déploiement (CI/CD) et gère l'infrastructure.
- Le/La **QA/Testeur** ✓ conçoit les tests (unitaires, intégration, E2E) et valide la qualité.
- Le/La **Data Engineer** ✓ modélise les données et construit des pipelines performants.

💡 Dans un projet, une bonne répartition des tâches renforce la qualité, la vitesse de livraison et la satisfaction de l'équipe.

8 / 8